

MODUL PELATIHAN

Kontainerisasi Laravel hingga Produksi di Kubernetes

Aspek	Keterangan
Durasi	4 hari ($\pm$ 24 jam efektif)
Tingkat	Menengah
Prasyarat	Dasar Laravel dan baris perintah; tanpa pengalaman Docker/Kubernetes
Teknologi	Laravel 13 · PHP 8.4 · MySQL 8.4 · Redis 8 · Caddy 2 · Kubernetes 1.34

Daftar Isi

Daftar Isi.....	2
Pengantar.....	6
Target Peserta.....	6
Prasyarat.....	6
Tujuan Pelatihan.....	6
Lingkungan yang Dipakai.....	7
Cara Memakai Modul Ini.....	7
Satu Analogi untuk Semua Konsep.....	8
Hari 1 — Fondasi: dari Aplikasi Menjadi Image.....	9
1.1 Masalah "Jalan di Komputer Saya".....	9
Masalah.....	9
Konsep (sesi konseptual).....	9
1.2 Image, Layer, dan Cache Build.....	10
Masalah.....	10
Konsep.....	10
Praktik Terpandu.....	11
1.3 PHP-FPM dan Web Server: Kenapa Dua Proses.....	12
Masalah.....	12
Konsep.....	12
Praktik Terpandu.....	13
1.4 Multi-stage Build.....	13
Masalah.....	13
Konsep.....	13
Praktik Terpandu.....	14
1.5 Build Context, .dockerignore, dan Rahasia yang Tidak Sengaja Terbawa.....	15
Masalah.....	15
Konsep.....	15
Praktik Terpandu.....	16
Hari 2 — Stack Lengkap dengan Docker Compose.....	16
2.1 Compose: Service, Network, dan Volume.....	17
Masalah.....	17
Konsep.....	17
Praktik Terpandu.....	18

2.2 MySQL: Konfigurasi, Healthcheck, dan Inisialisasi Sekali Seumur Hidup.....	19
Masalah.....	19
Konsep.....	19
Praktik Terpandu.....	19
2.3 Redis: Dua Instance dan Kebijakan Eviction	20
Masalah.....	20
Konsep.....	20
Praktik Terpandu.....	21
2.4 Queue, Horizon, dan Scheduler	22
Masalah.....	22
Konsep.....	22
Praktik Terpandu.....	22
2.5 Entrypoint: Urutan Boot, Cache Konfigurasi, dan Migrasi.....	23
Masalah.....	23
Konsep.....	23
Praktik Terpandu.....	24
2.6 Caddy dan Nginx: Dua Jalan Menuju Hasil yang Sama	24
Masalah.....	24
Konsep.....	24
Praktik Terpandu.....	25
2.7 Mode Pengembangan: Bind Mount, Vite, dan Filesystem Windows	26
Masalah.....	26
Konsep.....	26
Praktik Terpandu.....	27
Hari 3 — Kubernetes: Konsep Inti.....	27
3.1 Kenapa Compose Tidak Cukup.....	27
Masalah.....	27
Konsep (sesi konseptual)	27
3.2 Pod, Deployment, ReplicaSet, dan Service	29
Masalah.....	29
Konsep.....	29
Praktik Terpandu.....	30
3.3 ConfigMap dan Secret.....	30
Masalah.....	30

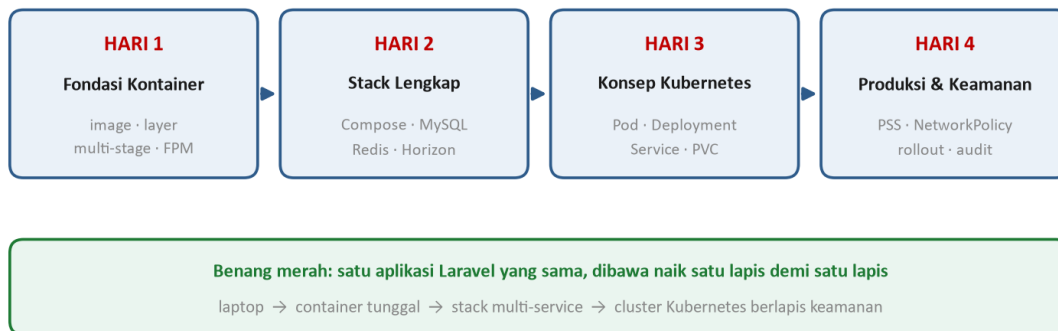
Konsep.....	30
Praktik Terpandu.....	31
3.4 Penyimpanan: PVC, StatefulSet, dan Akses Bersama	32
Masalah.....	32
Konsep.....	32
Praktik Terpandu.....	32
3.5 Probe: Startup, Readiness, dan Liveness	33
Masalah.....	33
Konsep.....	33
Praktik Terpandu.....	34
3.6 Kustomize: Base, Overlay, dan Generator	34
Masalah.....	34
Konsep.....	35
Praktik Terpandu.....	35
Hari 4 — Produksi dan Keamanan	36
4.1 Pod Security Admission.....	36
Masalah.....	36
Konsep.....	36
Praktik Terpandu.....	37
4.2 securityContext: Non-root dan Rootfs Read-only.....	38
Masalah.....	38
Konsep.....	38
Praktik Terpandu.....	39
4.3 NetworkPolicy dan Batas Kepercayaan	39
Masalah.....	39
Konsep.....	39
Praktik Terpandu.....	40
4.4 Migrasi Database sebagai Job	41
Masalah.....	41
Konsep.....	41
Praktik Terpandu.....	41
4.5 Rilis Tanpa Downtime	42
Masalah.....	42
Konsep.....	42

Praktik Terpandu.....	43
4.6 Autentikasi dan Gate Dashboard	43
Masalah.....	43
Konsep.....	43
Praktik Terpandu.....	44
4.7 Membaca Kegagalan: Studi Kasus Nyata	44
Masalah.....	44
Konsep (sesi konseptual)	44
Kasus 1 — exec: operation not permitted	44
Kasus 2 — Host ... is not allowed to connect.....	45
Kasus 3 — Menghapus PVC tidak menghapus data.....	45
Kasus 4 — Aturan penolakan yang kalah urutan	45
Lampiran A — Pemetaan Silabus ke Sesi	45
Lampiran B — Pemetaan Praktik Terpandu ke Latihan Mandiri.....	47
Lampiran C — Berkas Pendukung.....	48
C.1 Job Contoh untuk Menguji Antrean.....	48
C.2 Perintah yang Paling Sering Dipakai	49
C.3 Urutan Diagnosis Saat Pod Bermasalah.....	49

Pengantar

Modul ini membawa satu aplikasi Laravel 13 dari berkas di laptop sampai berjalan di cluster Kubernetes dengan postur keamanan yang ketat. Tidak ada lompatan: setiap lapisan dibangun di atas lapisan sebelumnya, dan setiap keputusan teknis dijelaskan alasannya — bukan sekadar disalin.

Dari laptop sampai produksi



Gambar 1. Peta empat hari pelatihan.

Target Peserta

Pengembang PHP/Laravel, DevOps engineer, atau system administrator yang sudah terbiasa menjalankan aplikasi Laravel secara langsung di server, dan ingin memindahkannya ke container lalu ke Kubernetes.

Prasyarat

- Terbiasa dengan baris perintah (PowerShell atau bash).
- Memahami dasar Laravel: composer, artisan, migrasi, queue.
- Memahami dasar HTTP, port, dan proses.
- TIDAK diperlukan pengalaman Docker maupun Kubernetes sebelumnya.

Tujuan Pelatihan

Setelah menyelesaikan empat hari, peserta mampu:

- Menjelaskan perbedaan image, container, dan layer, serta dampaknya pada kecepatan build.
- Menulis Dockerfile multi-stage yang menghasilkan image produksi ramping dan non-root.
- Merakit stack multi-service dengan Docker Compose lengkap dengan healthcheck.
- Memetakan setiap konsep Compose ke objek Kubernetes yang setara.
- Men-deploy stack ke Kubernetes memakai Kustomize.
- Menerapkan Pod Security restricted, securityContext, dan NetworkPolicy.
- Membaca kegagalan nyata dari log dan status, lalu memperbaikinya.

Lingkungan yang Dipakai

Komponen	Versi	Catatan
Laravel	13.x	PHP ^8.3
PHP	8.4 (Alpine)	php-fpm, ekstensi via install-php-extensions
MySQL	8.4 LTS	beberapa opsi lama sudah dihapus di versi ini
Redis	8.x (Alpine)	dua instance terpisah
Caddy	2.x	web server utama; nginx tersedia sebagai alternatif
Docker Desktop	Docker 29.x	Compose v5, Kubernetes v1.34
Kubernetes	1.34 (Docker Desktop)	satu node, CNI bawaan

Cara Memakai Modul Ini

Setiap sesi disusun dalam empat babak yang selalu sama urutannya. Pola ini sengaja diulang agar peserta tahu apa yang akan datang dan fasilitator punya ritme yang konsisten.

Babak	Isi	Peran fasilitator
Masalah	Situasi nyata yang bikin repot bila tidak ada konsep ini.	Ajukan sebagai pertanyaan, jangan langsung dijawab.
Konsep	Penjelasan, diagram, dan satu analogi dapur.	Papan tulis; pastikan analogi tersambung ke istilah teknis.
Praktik Terpandu	Perintah bernomor [1]..[n] plus 'Yang harus terlihat'.	Peserta mengetik sendiri; fasilitator berkeliling.
Kesalahan Umum	Jebakan yang paling sering menimpa pemula.	Bacakan SEBELUM peserta terjebak, bukan sesudah.

Sesi yang tidak memiliki perintah untuk dijalankan ditandai "Konsep (sesi konseptual)" dan tidak memuat babak Praktik Terpandu.

Satu Analogi untuk Semua Konsep

Sepanjang modul dipakai satu dunia analogi yang sama: dapur restoran. Konsep baru selalu disambungkan ke dunia yang sudah dikenal peserta, sehingga istilah teknis tidak menumpuk sebagai hafalan lepas.

Istilah teknis	Padanan di dapur
Image	Resep baku yang tertulis dan tidak berubah
Container	Satu porsi masakan yang sedang dimasak dari resep itu
Layer	Tahap persiapan yang bisa dipakai ulang, mis. kaldu yang sudah jadi
Volume	Gudang bahan di luar dapur — isinya tetap walau dapur dibongkar
Docker Compose	Satu dapur berisi beberapa stasiun kerja
Kubernetes	Jaringan restoran dengan manajer pusat
Pod	Satu meja kerja (bisa memuat lebih dari satu alat)
Deployment	SOP: "stasiun ini harus selalu diisi 2 koki"
Service	Nomor loket pemesanan yang tidak pernah berubah
Secret	Brankas resep rahasia
Probe	Kepala dapur yang berkeliling dan bertanya
NetworkPolicy	Kartu akses: siapa boleh masuk ruangan mana

VERSI DAN KETERANGAN

Modul versi 1.0 — Juli 2026.

Seluruh perintah, keluaran, dan angka dalam modul ini diambil dari sesi kerja nyata di Docker Desktop pada Windows 11, bukan dari dokumentasi.

Bila keluaran di layar peserta berbeda, dahulukan yang terlihat di layar — lalu cari tahu apa yang berbeda. Itu justru bahan belajar terbaik.

Hari 1 — Fondasi: dari Aplikasi Menjadi Image

TUJUAN PEMBELAJARAN HARI 1

- Menjelaskan perbedaan container dan mesin virtual, serta kapan masing-masing tepat.
- Menjelaskan image sebagai tumpukan layer, dan mengurutkan Dockerfile agar cache bekerja.
- Membangun image Laravel 13 yang berjalan dengan php-fpm.
- Memisahkan tahap build dan tahap runtime memakai multi-stage build.
- Menghasilkan image produksi tanpa Composer, tanpa Node, dan tidak berjalan sebagai root.

1.1 Masalah "Jalan di Komputer Saya"

Masalah

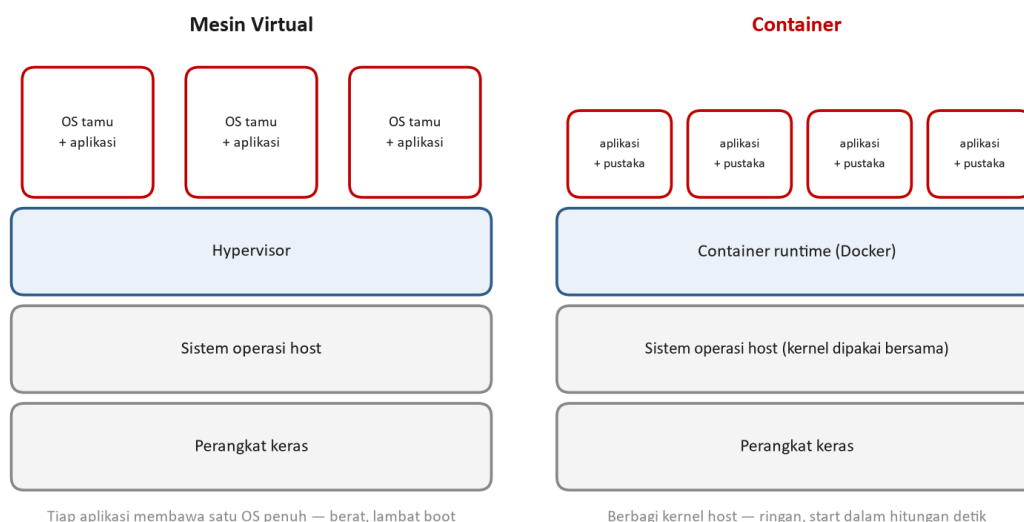
Sebuah aplikasi Laravel berjalan mulus di laptop pengembang. Dipindah ke server, aplikasi gagal: versi PHP beda tipis, satu ekstensi tidak terpasang, zona waktu tidak sama, dan sebuah paket sistem punya versi berbeda. Tim menghabiskan satu hari penuh menyamakan server dengan laptop. Bulan depan ada server baru, dan hari itu terulang.

Akar masalahnya bukan kecerobohan. Aplikasi selama ini dikirim tanpa lingkungannya. Yang berpindah hanya kode; segala hal yang dibutuhkan kode untuk hidup ditinggal di belakang, lalu disusun ulang secara manual di tempat tujuan — dan penyusunan ulang manual selalu meleset.

Konsep (sesi konseptual)

Container menyelesaikan ini dengan mengirim aplikasi bersama lingkungannya sebagai satu paket: sistem berkas berisi PHP, ekstensi, pustaka sistem, dan kode aplikasi. Yang tidak ikut hanyalah kernel — container memakai kernel milik sistem operasi host.

Di sinilah container berbeda dari mesin virtual. Mesin virtual mensimulasikan perangkat keras dan menjalankan sistem operasi tamu yang utuh, sehingga tiap aplikasi membawa satu OS penuh: berat, boot dalam hitungan menit. Container hanya mengisolasi proses memakai fitur kernel (namespace dan cgroup), sehingga ukurannya megabyte dan start dalam hitungan detik.



Gambar 2. Mesin virtual membawa OS tamu sendiri; container berbagi kernel host.

Analogi — mesin virtual seperti menyewa satu bangunan restoran lengkap untuk tiap menu; container seperti satu stasiun kerja di dapur bersama — punya alat dan bahannya sendiri, tapi listrik, air, dan gedungnya dipakai bersama.

Konsekuensi yang paling sering disalahpahami: karena kernel dipakai bersama, container Linux tidak bisa berjalan di atas kernel Windows secara langsung. Docker Desktop menjalankan sebuah mesin virtual Linux kecil (WSL2) di belakang layar, dan seluruh container berjalan di dalamnya. Ini bukan detail sepele: kelak ia menjelaskan kenapa akses berkas dari drive Windows terasa lambat.

KESALAHAN UMUM PEMULA

Menganggap container adalah "VM yang ringan". Container bukan mesin — ia proses yang diisolasi. Tidak ada proses init, tidak ada systemd, dan ketika proses utamanya berhenti, container ikut berhenti.

Menaruh beberapa layanan dalam satu container agar "praktis". Akibatnya log tercampur, dan kematian salah satu proses tidak terdeteksi karena container masih dianggap hidup.

1.2 Image, Layer, dan Cache Build

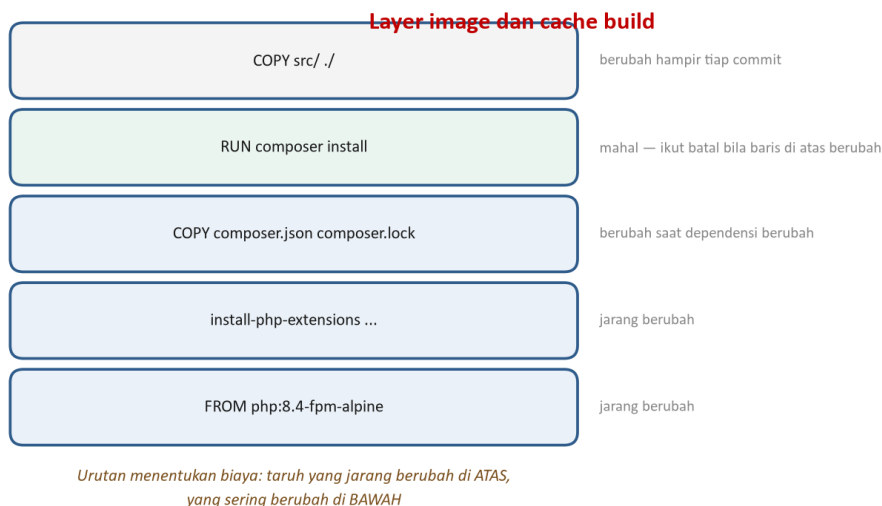
Masalah

Build image pertama kali memakan lima menit — masih wajar. Tetapi setelah mengubah satu baris kode PHP, build berikutnya tetap lima menit: Composer mengunduh ulang seluruh dependensi. Dalam sehari peserta melakukan puluhan build, dan waktu tunggu menjadi biaya nyata.

Konsep

Image bukan satu berkas besar, melainkan tumpukan layer yang hanya-baca. Setiap instruksi di Dockerfile menghasilkan satu layer berisi selisih perubahan terhadap layer sebelumnya. Container adalah image tersebut ditambah satu layer tulis tipis di paling atas; menghapus container hanya membuang layer tipis itu.

Docker menyimpan cache tiap layer. Saat build ulang, layer dipakai ulang selama instruksinya sama dan seluruh layer di bawahnya juga tidak berubah. Kalimat terakhir itu kuncinya: satu layer yang berubah membatalkan cache SEMUA layer di atasnya, tanpa peduli isinya sama atau tidak.



Gambar 3. Urutan instruksi menentukan berapa banyak pekerjaan yang harus diulang.

Analogi — layer seperti tahap persiapan di dapur. Kaldu yang sudah direbus bisa dipakai untuk banyak menu; tapi kalau resep memutuskan mengganti air di langkah paling awal, semua tahap sesudahnya harus diulang.

Dari sini lahir satu aturan praktis yang menentukan hampir seluruh struktur Dockerfile: letakkan yang jarang berubah di atas, yang sering berubah di bawah. Karena itu manifest dependensi disalin dan dipasang lebih dulu, baru kode aplikasi disalin belakangan.

Dockerfile – urutan menentukan biaya

```
# SALAH – kode disalin sebelum dependensi dipasang
COPY src/ ./          # berubah tiap commit
RUN composer install   # ikut diulang tiap commit, walau dependensi sama

# BENAR – manifest dulu, kode belakangan
COPY src/composer.json src/composer.lock ./
RUN composer install   # hanya diulang bila composer.lock berubah
COPY src/ ./
```

Praktik Terpandu

Membuktikan perilaku cache secara langsung.

- [1] Siapkan direktori kerja dan proyek Laravel 13
`docker run --rm -v "${PWD}:/app" -w /app composer:2 \`
`create-project laravel/laravel:^13.0 src --no-interaction --no-scripts`
- [2] Lihat daftar image yang sekarang ada di mesin
`docker images`
- [3] Bangun sebuah image percobaan dua kali berturut-turut
`docker build -t uji-cache -f docker/php/Dockerfile --target base .`
`docker build -t uji-cache -f docker/php/Dockerfile --target base .`

Yang harus terlihat:

- Build pertama menampilkan setiap langkah dengan durasi.
- Build kedua menampilkan CACHED pada hampir seluruh langkah dan selesai dalam hitungan detik.

- Bila sebuah berkas yang disalin di langkah awal diubah, CACHED hilang dari titik itu ke bawah.

KESALAHAN UMUM PEMULA

Menyangka `docker build` selalu mengunduh ulang semuanya, lalu menambahkan `--no-cache` "supaya bersih". Itu justru membuang satu-satunya mekanisme yang mempercepat build.

Menaruh COPY seluruh direktori di baris paling atas. Semua layer sesudahnya jadi tidak pernah kena cache — gejalanya: build selalu lama, tanpa alasan yang jelas.

1.3 PHP-FPM dan Web Server: Kenapa Dua Proses

Masalah

Peserta yang terbiasa dengan `php artisan serve` bertanya wajar: kenapa produksi butuh dua container — satu web server dan satu PHP — padahal satu perintah saja sudah bisa melayani halaman?

Konsep

`php artisan serve` memakai web server bawaan PHP yang bersifat satu proses dan tidak dirancang untuk beban nyata. Di produksi, PHP dijalankan sebagai PHP-FPM (FastCGI Process Manager): sekumpulan proses pekerja yang siap sedia menunggu request, dikelola oleh satu proses master.

PHP-FPM tidak berbicara HTTP. Ia berbicara protokol FastCGI dan mendengarkan di port 9000. Yang berbicara HTTP dengan browser adalah web server — Caddy atau nginx — yang meneruskan request dinamis ke PHP-FPM lalu mengembalikan hasilnya.

Pembagian ini bukan sekadar tradisi. Berkas statis — CSS, JavaScript, gambar, berkas unggahan — dilayani web server langsung dari disk tanpa pernah menyentuh PHP. Pada aplikasi nyata, sebagian besar request adalah berkas statis; membiarkan PHP menanganinya berarti membuang proses pekerja yang mahal untuk pekerjaan yang murah.

Analogi — *web server adalah pelayan di depan: ia sendiri yang mengambil air putih dan tisu dari rak (berkas statis), dan hanya meneruskan pesanan masakan (request dinamis) ke dapur.*

Aspek	<code>php artisan serve</code>	PHP-FPM + web server
Jumlah proses	satu	master + banyak pekerja
Request bersamaan	satu per satu	sebanyak <code>pm.max_children</code>
Berkas statis	dilayani PHP	dilayani web server dari disk
Peruntukan	pengembangan	produksi

Praktik Terpandu

- [1] Jalankan container php-fpm dan lihat proses di dalamnya

```
docker run -d --name coba-fpm php:8.4-fpm-alpine
docker exec coba-fpm ps -o user,args
```
- [2] Perhatikan pembagian hak akses master dan pekerja
(master berjalan sebagai root, pool www sebagai www-data)
- [3] Bersihkan

```
docker rm -f coba-fpm
```

Yang harus terlihat: satu baris "php-fpm: master process" milik root, dan beberapa baris "php-fpm: pool www" milik www-data.

KESALAHAN UMUM PEMULA

Mencoba membuka `http://localhost:9000` di browser lalu bingung karena kosong. Port 9000 bukan HTTP — ia FastCGI. Yang dibuka browser adalah port web server.

Menyalin berkas `public/` hanya ke container PHP. Web server punya salinan sendiri; kalau tidak ikut disalin, seluruh CSS dan JavaScript menjadi 404.

1.4 Multi-stage Build

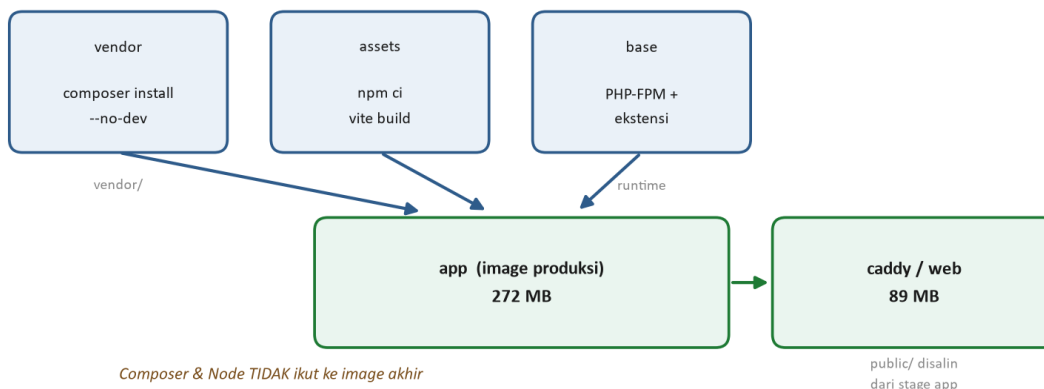
Masalah

Untuk membangun aplikasi dibutuhkan Composer dan Node.js. Bila keduanya dipasang di image yang sama dengan yang dikirim ke produksi, hasilnya image berukuran lebih dari satu gigabyte yang membawa serta compiler, package manager, dan seluruh cache-nya. Semua itu tidak dipakai saat aplikasi melayani request, tetapi tetap ikut diunduh setiap kali deploy — dan tetap bisa dipakai penyerang yang berhasil masuk.

Konsep

Multi-stage build memisahkan tahap membangun dari tahap menjalankan. Satu Dockerfile boleh berisi beberapa stage; tiap stage punya image dasarnya sendiri. Stage terakhir menyalin hanya hasil jadi dari stage-stage sebelumnya, dan seluruh perkakas yang dipakai untuk membuatnya ditinggalkan.

Multi-stage build



Gambar 4. Perangkat build tinggal di stage-nya; hanya hasil jadi yang naik ke image akhir.

Analogi — dapur persiapan dan dapur saji dipisah. Panci besar, mesin pengaduk, dan karung tepung tidak ikut keluar ke ruang makan — yang keluar hanya piring yang sudah siap.

docker/php/Dockerfile — kerangka multi-stage

```
# syntax=docker/dockerfile:1.7
ARG PHP_VERSION=8.4

FROM php:${PHP_VERSION}-fpm-alpine AS base
COPY --from=mlocati/php-extension-installer:2 \
  /usr/bin/install-php-extensions /usr/local/bin/
RUN install-php-extensions bcmath exif gd intl opcache \
  pcntl pdo_mysql posix redis zip
WORKDIR /var/www/html

FROM base AS vendor
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY src/composer.json src/composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist
COPY src/ ./
RUN composer dump-autoload --no-dev --optimize --classmap-authoritative

FROM node:22-alpine AS assets
WORKDIR /app
COPY src/package*.json ./
RUN npm ci
COPY src/resources ./resources
COPY src/vite.config.js ./
RUN npm run build

FROM base AS app
COPY --chown=www-data:www-data src/ /var/www/html/
COPY --from=vendor --chown=www-data:www-data /app/vendor /var/www/html/vendor
COPY --from=assets --chown=www-data:www-data /app/public/build /var/www/html/public/build
```

Perhatikan bahwa stage **vendor** dibangun DI ATAS **base**, bukan di atas image Composer resmi. Alasannya penting: Composer memeriksa versi PHP dan ekstensi yang tersedia saat menyelesaikan dependensi. Bila dipasang memakai PHP versi lain, hasilnya bisa berbeda dengan yang nanti benar-benar dijalankan.

Praktik Terpandu

[1] Bangun hanya sampai stage tertentu untuk memeriksa isinya

```
docker build -t coba-vendor -f docker/php/Dockerfile --target vendor .

[2] Bangun image produksi
docker build -t laravel-app:latest -f docker/php/Dockerfile --target app .

[3] Bandingkan ukuran keduanya
docker images | Select-String "coba-vendor|laravel-app"

[4] Buktikan perkakas build TIDAK ikut ke image produksi
docker run --rm --entrypoint sh laravel-app:latest \
  -c "command -v composer || echo TIDAK ADA"
docker run --rm --entrypoint sh laravel-app:latest \
  -c "command -v npm || echo TIDAK ADA"
```

Yang harus terlihat: image produksi jauh lebih kecil, dan kedua perintah di langkah [4] menjawab TIDAK ADA.

KESALAHAN UMUM PEMULA

Lupa `--from=<stage>` pada COPY, sehingga yang tersalin adalah berkas dari build context, bukan hasil stage sebelumnya. Gejalanya: vendor/ kosong walau stage vendor sukses.

Menjalankan `composer install` di stage akhir "biar sekalian". Composer kembali ikut ke image produksi, dan seluruh manfaat multi-stage hilang.

1.5 Build Context, .dockerignore, dan Rahasia yang Tidak Sengaja Terbawa

Masalah

Perintah `docker build .` mengirim seluruh isi direktori ke daemon Docker sebelum satu instruksi pun dijalankan. Bila direktori itu memuat `vendor/`, `node_modules/`, riwayat `.git`, dan berkas `.env` berisi password produksi, maka semuanya ikut terkirim — dan yang tersalin oleh `COPY . .` akan ikut tertanam di dalam image.

Berkas `.env` di dalam image adalah masalah nyata, bukan teoretis. Image biasanya dikirim ke registry dan bisa ditarik banyak orang. Siapa pun yang bisa menarik image itu bisa membaca seluruh isinya, termasuk password database.

Konsep

`.dockerignore` menentukan apa yang TIDAK dikirim ke build context. Ia bekerja mirip `.gitignore`, tetapi dampaknya berbeda: bukan soal kerapian repositori, melainkan soal kecepatan build dan permukaan serangan image.

Analogi — *build context adalah troli yang didorong masuk ke dapur. Tidak semua isi gudang perlu ikut; dan brankas resep rahasia jelas tidak boleh ikut naik ke troli.*

.dockerignore

```
.git
.env
.env.*
!.env.example

# Dependensi dibangun ulang di stage vendor/assets
src/vendor
src/node_modules
src/public/build
```

```
src/public/hot

# Artefak runtime
src/storage/framework/cache/*
src/storage/logs/*
src/bootstrap/cache/*
```

Baris `src/public/hot` layak diperhatikan. Berkas itu dibuat Vite saat mode pengembangan dan berisi alamat dev server. Bila ikut masuk ke image produksi, aplikasi akan mencoba memuat aset dari `localhost:5173` milik pengembang — dan seluruh tampilan rusak di server.

Praktik Terpandu

- [1] Lihat berapa besar build context yang terkirim
`docker build -f docker/php/Dockerfile --target base -t uji .`
 (perhatikan baris "transferring context" di awal keluaran)
- [2] Pastikan `.env` TIDAK ada di dalam image produksi
`docker run --rm --entrypoint sh laravel-app:latest \`
`-c "ls -a /var/www/html | grep -c env"`
- [3] Periksa isi image lapis demi lapis
`docker history laravel-app:latest`

Yang harus terlihat: ukuran context kecil (satuan megabyte, bukan ratusan), dan langkah [2] tidak menemukan berkas `.env`.

CATATAN FASILITATOR

Sesi ini adalah titik yang tepat untuk menegaskan bahwa keamanan bukan bab terakhir pelatihan. Keputusan di hari pertama — apa yang ikut ke dalam image — sudah menentukan seberapa besar kerusakan bila kelak ada yang bocor.

Bila waktu memungkinkan, tunjukkan `docker history` pada image yang dibangun tanpa `.dockerignore` sebagai pembanding.

Hari 2 — Stack Lengkap dengan Docker Compose

TUJUAN PEMBELAJARAN HARI 2

- Merakit stack multi-service memakai Docker Compose.
- Menjelaskan peran healthcheck dan urutan boot antar service.
- Menjelaskan mengapa cache dan queue memakai instance Redis yang berbeda.
- Menjalankan queue worker dengan Horizon dan scheduler sebagai pengganti cron.
- Menulis entrypoint yang idempoten dan aman di-restart.
- Memisahkan konfigurasi pengembangan dari produksi tanpa menduplikasi berkas.

2.1 Compose: Service, Network, dan Volume

Masalah

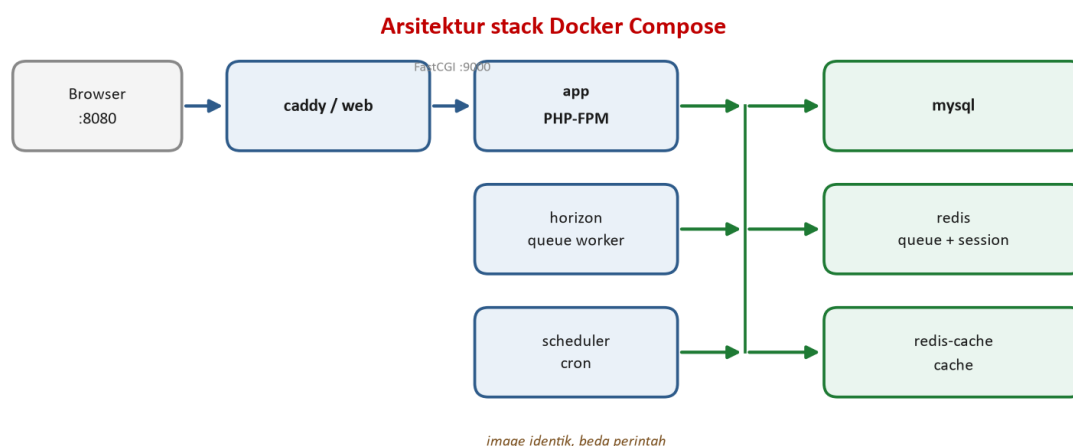
Aplikasi sekarang berjalan di satu container. Tetapi ia butuh database, penyimpanan cache, pemroses antrean, dan web server. Menjalankan lima perintah `docker run` dengan belasan flag setiap pagi, dalam urutan yang benar, dengan jaringan yang saling terhubung, jelas tidak berkelanjutan — dan tidak bisa diserahkan ke orang lain.

Konsep

Docker Compose memindahkan seluruh perintah itu ke satu berkas YAML yang bisa dibaca, ditinjau, dan disimpan bersama kode. Tiga konsep dasarnya:

Konsep	Fungsi	Padanan perintah manual
service	Satu container beserta konfigurasinya	satu <code>docker run</code>
network	Jaringan privat tempat service saling memanggil lewat NAMA	<code>docker network create + --network</code>
volume	Penyimpanan yang hidup lebih lama daripada container	<code>-v nama:/path</code>

Yang paling sering luput: di dalam network Compose, nama service adalah nama host. Aplikasi menghubungi database dengan alamat `mysql`, bukan alamat IP. Docker menyediakan DNS internal yang menerjemahkannya, dan alamat IP boleh berubah kapan saja tanpa mengganggu aplikasi.



/build/ dan /storage/* dilayani web server langsung dari disk — tidak menyentuh PHP*

Gambar 5. Stack lengkap: satu jalur request, tiga workload PHP, tiga penyimpanan.

Analogi — *Compose adalah denah satu dapur: tiap stasiun kerja punya tugasnya, ada jalur pelayan yang menghubungkan antar stasiun, dan ada gudang yang isinya tidak ikut hilang saat stasiun dibongkar.*

compose.yaml — kerangka minimal

```
name: laravel-app

services:
  app:
    build:
      context: .
      dockerfile: docker/php/Dockerfile
      target: app
    restart: unless-stopped
    env_file: [.env]
    depends_on:
      mysql:
        condition: service_healthy
    networks: [backend]

volumes:
  mysql-data:

networks:
  backend:
    driver: bridge
```

Praktik Terpandu

- [1] Jalankan seluruh stack
docker compose up -d --build
- [2] Lihat status dan kesehatan tiap service
docker compose ps
- [3] Buktikan resolusi nama antar service bekerja
docker compose exec app getent hosts mysql
docker compose exec app getent hosts redis
- [4] Lihat log satu service tertentu
docker compose logs -f app

Yang harus terlihat: kolom STATUS menampilkan (healthy) untuk service yang punya healthcheck, dan langkah [3] mengembalikan alamat IP internal untuk tiap nama.

KESALAHAN UMUM PEMULA

Memakai `localhost` sebagai `DB_HOST` di dalam container. Di dalam container, `localhost` adalah container itu sendiri — bukan mesin host, dan bukan container database.

Menganggap `depends_on` menunggu service siap. Tanpa `condition: service_healthy`, ia hanya menunggu container DIMULAI — bukan menunggu database siap menerima koneksi.

2.2 MySQL: Konfigurasi, Healthcheck, dan Inisialisasi Sekali Seumur Hidup

Masalah

Container database berstatus "Up" beberapa detik setelah dijalankan, tetapi aplikasi yang langsung menyambung tetap gagal. MySQL memang sudah menyala, namun belum selesai menyiapkan direktori data, membuat user aplikasi, dan membuka port. "Up" tidak sama dengan "siap".

Konsep

Healthcheck adalah perintah yang dijalankan Docker secara berkala di dalam container untuk menjawab satu pertanyaan: apakah layanan ini benar-benar bisa dipakai? Statusnya (healthy / unhealthy) dapat dipakai service lain lewat `depends_on: condition: service_healthy`.

Kualitas healthcheck menentukan kualitas informasinya. Memeriksa apakah port terbuka jauh lebih lemah daripada menjalankan query nyata sebagai user aplikasi — dan perbedaan itu akan terasa mahal pada Hari 4.

compose.yaml – service mysql

```
mysql:
  build: { context: ./docker/mysql }
  environment:
    MYSQL_DATABASE: ${DB_DATABASE}
    MYSQL_USER: ${DB_USERNAME}
    MYSQL_PASSWORD: ${DB_PASSWORD:?wajib diisi di .env}
    MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?wajib diisi di .env}
    MYSQL_ROOT_HOST: localhost
  volumes:
    - mysql-data:/var/lib/mysql
  healthcheck:
    test: ["CMD-SHELL", 'mysqladmin ping -h 127.0.0.1 -u"${MYSQL_USER}" \
      -p"${MYSQL_PASSWORD}" --silent']
    interval: 10s
    retries: 12
    start_period: 40s
```

Sintaks `${DB_PASSWORD:?pesan}` membuat Compose menolak berjalan bila variabel itu kosong. Lebih baik gagal terang-terangan saat start daripada database menyala dengan password kosong.

Analogi — *healthcheck adalah kepala dapur yang mencicipi masakan sebelum dikirim ke meja — bukan sekadar melihat apakah kompornya menyala.*

Satu sifat MySQL yang wajib dipahami sejak awal: inisialisasi hanya terjadi SEKALI, yaitu saat direktori data masih kosong. Di saat itulah user aplikasi dibuat dan password root ditetapkan. Bila proses ini terputus di tengah, direktori data berisi tabel sistem tetapi tanpa user aplikasi — dan MySQL akan start normal selamanya sesudahnya, karena direktorinya sudah tidak kosong lagi. Gejalanya menyesatkan: aplikasi ditolak dengan pesan "Host ... is not allowed to connect", padahal database terlihat sehat.

Praktik Terpandu

- [1] Amati proses inisialisasi dari nol


```
docker compose down -v
docker compose up -d mysql
docker compose logs -f mysql
```
- [2] Cari tiga baris penanda inisialisasi berhasil


```
docker compose logs mysql | \
  Select-String "Initializing database|Temporary server|initialized"
```

```
[3] Uji koneksi sebagai user aplikasi, bukan root
docker compose exec mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" \
-D"$MYSQL_DATABASE" -e "SELECT 1"'
```

Yang harus terlihat: baris "Initializing database files", lalu "Temporary server started", lalu "Database files initialized". Bila ketiganya tidak muncul pada volume yang baru, inisialisasi tidak berjalan sebagaimana mestinya.

KESALAHAN UMUM PEMULA

Mengubah password di .env lalu heran karena tidak berlaku. Password hanya dipasang saat inisialisasi pertama; volume lama tetap memakai password lama.

Menghapus container database lalu menyangka datanya ikut hilang. Data ada di volume — `docker compose down` saja tidak menghapusnya, perlu `down -v`.

Memakai `mysqladmin ping` sebagai satu-satunya healthcheck. Perintah itu keluar dengan status sukses BAHKAN ketika autentikasi ditolak, sehingga database yang rusak tetap dilaporkan sehat.

2.3 Redis: Dua Instance dan Kebijakan Eviction

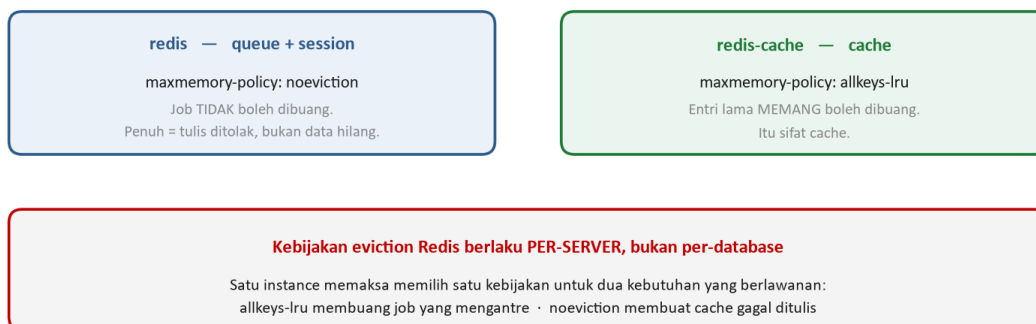
Masalah

Aplikasi memakai Redis untuk dua hal sekaligus: menyimpan cache dan menampung antrian job. Keduanya sama-sama data di memori, jadi wajar bila peserta menaruhnya di satu instance. Suatu hari memori Redis penuh, dan job yang sudah masuk antrian menghilang tanpa jejak — tanpa error, tanpa log.

Konsep

Ketika memori mencapai batas `maxmemory`, Redis bertindak sesuai `maxmemory-policy`. Kuncinya: kebijakan itu berlaku PER-SERVER, bukan per-database. Memisahkan cache ke database 1 dan queue ke database 0 dalam satu instance TIDAK memisahkan kebijakannya.

Kenapa Redis harus dua instance



Gambar 6. Satu instance memaksa satu kebijakan untuk dua kebutuhan yang berlawanan.

Kebijakan	Perilaku saat penuh	Cocok untuk	Bahaya bila salah tempat
allkeys-lru	Membuang key yang paling lama tidak dipakai	cache	Job yang mengantre ikut dibuang — hilang diam-diam
noeviction	Menolak penulisan baru (error OOM)	queue, session	Penulisan cache mulai gagal saat memori penuh

Analogi — cache adalah rak bumbu di meja kerja — kalau penuh, yang jarang dipakai boleh disingkirkan. Antrean pesanan adalah papan order — tidak boleh ada satu pun kertas yang dibuang hanya karena papannya penuh.

Karena itu stack ini memakai dua instance: `redis` untuk queue dan session dengan `noeviction` serta persistensi AOF, dan `redis-cache` untuk cache dengan `allkeys-lru` tanpa persistensi. Laravel mendukung ini lewat koneksi Redis terpisah di `config/database.php`.

src/config/database.php — koneksi cache menunjuk instance sendiri

```
'cache' => [
    'host' => env('REDIS_CACHE_HOST', env('REDIS_HOST', '127.0.0.1')),
    'password' => env('REDIS_CACHE_PASSWORD', env('REDIS_PASSWORD')),
    'port' => env('REDIS_CACHE_PORT', env('REDIS_PORT', '6379')),
    'database' => env('REDIS_CACHE_DB', '1'),
],
```

Pola `env('REDIS_CACHE_HOST', env('REDIS_HOST', ...))` membuat konfigurasi ini tetap bekerja bila kelak hanya tersedia satu instance: nilainya otomatis jatuh ke `REDIS_HOST`.

Praktik Terpandu

- Periksa kebijakan masing-masing instance


```
docker compose exec redis sh -c 'redis-cli --no-auth-warning \
-a "$REDIS_PASSWORD" config get maxmemory-policy'
docker compose exec redis-cache sh -c 'redis-cli --no-auth-warning \
-a "$REDIS_PASSWORD" config get maxmemory-policy'
```
- Tulis satu entri cache dari aplikasi


```
docker compose exec app php artisan tinker \
--execute="Cache::put('uji','ok',60); echo Cache::get('uji');"
```
- Buktikan entri itu mendarat di instance cache, bukan di instance queue


```
docker compose exec redis-cache sh -c 'redis-cli -n 1 keys "*"
docker compose exec redis sh -c 'redis-cli -n 0 keys "laravel_horizon*"'
```

Yang harus terlihat: instance `redis` menjawab `noeviction`, `redis-cache` menjawab `allkeys-lru`, dan kunci cache hanya muncul di `redis-cache`.

KESALAHAN UMUM PEMULA

Menyangka `REDIS_DB` yang berbeda sudah cukup memisahkan. Nomor database memisahkan ruang nama, bukan kebijakan eviction maupun batas memori.

Membiarkan `maxmemory` tidak disetel sama sekali. Redis akan tumbuh sampai memori host habis, dan yang terbunuh bisa jadi justru proses lain.

2.4 Queue, Horizon, dan Scheduler

Masalah

Mengirim email, membuat PDF, atau memanggil API pihak ketiga di tengah request membuat pengguna menunggu. Pekerjaan seperti itu perlu dipindah ke latar belakang. Tetapi begitu dipindah, muncul pertanyaan baru: siapa yang menjalankannya, berapa banyak pekerja yang dibutuhkan, dan apa yang terjadi bila salah satu pekerja mati di tengah pekerjaan.

Konsep

Queue memisahkan "menerima pekerjaan" dari "mengerjakannya". Request web hanya menaruh job ke antrean lalu selesai; proses lain — worker — mengambil dan mengerjakannya.

Laravel Horizon adalah supervisor untuk queue berbasis Redis. Ia mengelola sendiri jumlah proses worker, melakukan auto-scaling, mencatat job gagal, dan menyediakan dashboard. Karena Horizon yang mengelola siklus hidup worker, perintahnya cukup satu: `php artisan horizon`.

Scheduler adalah hal yang berbeda dan sering tertukar. Ia pengganti cron: menjalankan tugas terjadwal. Di container, `php artisan schedule:work` berjalan sebagai proses yang terus hidup.

***Analogi** — queue adalah papan order; Horizon adalah kepala dapur yang mengatur berapa koki berdiri di papan itu dan mencatat pesanan yang gagal; scheduler adalah alarm yang berbunyi tiap jam untuk tugas rutin seperti membersihkan meja.*

Ketiga workload PHP — app, horizon, scheduler — memakai image yang PERSIS SAMA dan hanya berbeda perintah. Ini bukan penghematan, melainkan jaminan: mustahil ada perbedaan versi kode antara yang melayani request dan yang memproses antrean.

compose.yaml — horizon dan scheduler

```
horizon:
  <<: *app-base
  command: ["php", "artisan", "horizon"]
  stop_signal: SIGTERM
  stop_grace_period: 60s
  healthcheck:
    test: ["CMD", "php", "artisan", "horizon:status"]

scheduler:
  <<: *app-base
  command: ["php", "artisan", "schedule:work"]
```

`stop_grace_period: 60s` penting saat deploy. Horizon menangkap SIGTERM lalu menunggu job yang sedang berjalan selesai; tanpa tenggang waktu yang cukup, Docker akan membunuhnya di tengah pekerjaan.

Praktik Terpandu

- [1] Buat sebuah job sederhana
(berkas `src/app/Jobs/PingJob.php` — lihat Lampiran C)
- [2] Kirim job ke antrean
`docker compose exec app php artisan tinker \`
`--execute="App\Jobs\PingJob::dispatch();"`
- [3] Amati worker memprosesnya

```
docker compose logs horizon --tail 10
```

```
[4] Periksa status supervisor
docker compose exec app php artisan horizon:status
```

Yang harus terlihat: baris PingJob ... RUNNING lalu ... DONE dengan durasi milidetik, dan horizon:status menjawab "Horizon is running".

KESALAHAN UMUM PEMULA

Mengubah kode job lalu heran karena worker masih menjalankan versi lama. Worker memuat kode sekali saat start — worker harus di-restart setelah deploy.

Menjalankan lebih dari satu container scheduler. Setiap tugas terjadwal akan berjalan dua kali.

2.5 Entrypoint: Urutan Boot, Cache Konfigurasi, dan Migrasi

Masalah

Sebelum aplikasi siap melayani, ada beberapa langkah yang harus terjadi: direktori writable harus ada, database harus sudah bisa dihubungi, cache konfigurasi harus dibangun, dan migrasi harus dijalankan. Menjalankannya manual setiap kali container start jelas tidak mungkin.

Konsep

Entrypoint adalah skrip yang berjalan sebelum proses utama container. Syarat mutlaknya: idempoten — aman dijalankan berulang kali, karena container bisa di-restart kapan saja.

Satu keputusan di sini sering salah dan akibatnya sulit dilacak: `php artisan config:cache` menyimpan seluruh nilai environment ke dalam sebuah berkas. Bila dijalankan saat BUILD, yang tersimpan adalah environment milik mesin build — bukan milik server. Karena itu cache konfigurasi harus dibangun saat runtime, di entrypoint.

Analogi — *entrypoint adalah daftar persiapan buka dapur: nyalakan kompor, pastikan gudang terbuka, tempelkan menu hari ini. Daftar yang sama dijalankan tiap pagi, dan menjalankannya dua kali tidak merusak apa pun.*

docker/php/bin/entrypoint.sh — inti alurnya

```
#!/bin/sh
set -eu
cd /var/www/html

# 1. Direktori writable — volume baru datang dalam keadaan kosong
mkdir -p storage/framework/cache/data storage/framework/sessions \
    storage/framework/views storage/logs bootstrap/cache

# 2. Gagal terang-terangan bila kunci aplikasi belum ada
if [ -z "${APP_KEY:-}" ]; then
    echo "[entrypoint] FATAL: APP_KEY belum di-set."; exit 1
fi

# 3. Tunggu database benar-benar menerima koneksi
until php -r 'exit(@fsockopen(getenv("DB_HOST"), 3306) ? 0 : 1);'; do sleep 2; done

# 4. Cache dibangun di RUNTIME, bukan saat build
if [ "${APP_ENV}" = "local" ]; then
    php artisan optimize:clear
```

```
else
    php artisan config:cache && php artisan route:cache
    php artisan view:cache && php artisan event:cache
fi

exec "$@"
```

Praktik Terpandu

- [1] Amati entrypoint bekerja dari awal


```
docker compose down
docker compose up -d
docker compose logs app | Select-String "entrypoint"
```
- [2] Buktikan cache konfigurasi benar-benar terbentuk


```
docker compose exec app php artisan about --only=cache
```
- [3] Buktikan penjaga APP_KEY bekerja


```
docker compose run --rm -e APP_KEY= app php-fpm
```

Yang harus terlihat: pada langkah [2] baris Config, Events, Routes, dan Views berstatus CACHED; pada langkah [3] container berhenti dengan pesan FATAL yang jelas, bukan error PHP yang membingungkan.

KESALAHAN UMUM PEMULA

Menjalankan `config:cache` di dalam Dockerfile. Nilai environment mesin build ikut terpengang, dan aplikasi di server memakai konfigurasi yang salah tanpa pesan error apa pun.

Menjalankan migrasi di entrypoint SETIAP container. Bila ada dua replika, keduanya bermigrasi bersamaan. Migrasi cukup dijalankan satu titik saja.

2.6 Caddy dan Nginx: Dua Jalan Menuju Hasil yang Sama

Masalah

Materi di internet terbelah: sebagian memakai nginx, sebagian Caddy. Peserta perlu tahu apa yang sebenarnya berbeda, dan mana yang perlu dikuasai.

Konsep

Keduanya melakukan pekerjaan yang sama: melayani berkas statis dari disk dan meneruskan request dinamis ke PHP-FPM. Perbedaannya ada pada bawaan dan pada gaya konfigurasi.

Aspek	Nginx	Caddy
TLS	disiapkan sendiri	otomatis, termasuk perpanjangan
Panjang konfigurasi	± 150 baris	± 60 baris
Urutan direktif	sesuai urutan penulisan	ada urutan bawaan sendiri

Populasi di lapangan	sangat umum	makin banyak, belum sebanyak nginx
----------------------	-------------	------------------------------------

Baris ketiga adalah jebakan nyata dan bukan sekadar selera. Pada Caddy, direktif TIDAK dieksekusi menurut urutan penulisan — ada urutan bawaan. Sebuah aturan penolakan yang ditulis di atas bisa kalah oleh `file_server` yang ditulis di bawah, sehingga berkas yang seharusnya ditolak justru tersaji. Solusinya membungkus seluruh handler dalam blok `route`, yang menghormati urutan penulisan.

Caddyfile – blok `route` menjaga urutan

```
route {
    @dotfiles {
        path_regexp /\.
        not path /.well-known/*
    }
    respond @dotfiles 403

    handle_path /storage/* {
        root * /var/www/html/storage/app/public
        file_server
    }

    handle {
        php_fastcgi app:9000
        file_server
    }
}
```

Analogi — *nginx seperti resep yang dikerjakan berurutan dari atas ke bawah; Caddy seperti dapur yang punya SOP sendiri tentang urutan kerja — kecuali Anda menuliskannya sebagai satu rangkaian yang wajib diikuti.*

Praktik Terpandu

- [1] Jalankan dengan Caddy (bawaan)


```
docker compose up -d
```
- [2] Uji perilaku penting


```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/.htaccess
```
- [3] Pindah ke nginx: ubah `COMPOSE_PROFILES=nginx` di `.env`, lalu


```
docker compose down --remove-orphans
docker compose up -d
```
- [4] Ulangi langkah [2] dan bandingkan hasilnya

Yang harus terlihat: halaman utama 200 dan `/.htaccess` 403 pada KEDUA web server. Bila `/.htaccess` menjawab 200, aturan penolakan tidak bekerja.

KESALAHAN UMUM PEMULA

Berganti profil tanpa `--remove-orphans`. Compose tidak menghentikan service di luar profil aktif, sehingga web server lama tetap hidup dan port 8080 bentrok.

Menguji aturan penolakan hanya dengan berkas yang memang tidak ada. Berkas yang tidak ada menjawab 404 karena tidak ditemukan — bukan karena ditolak. Uji dengan berkas yang benar-benar

ada.

2.7 Mode Pengembangan: Bind Mount, Vite, dan Filesystem Windows

Masalah

Image produksi memanggang kode ke dalamnya. Saat mengembangkan, mengubah satu baris lalu menunggu build ulang jelas tidak masuk akal. Peserta butuh perubahan yang langsung terlihat — tanpa mengorbankan kesamaan dengan produksi.

Konsep

Bind mount memetakan direktori host ke dalam container, sehingga container membaca berkas yang sama dengan yang diedit. `compose.override.yaml` dimuat otomatis oleh Compose dan menimpa definisi di `compose.yaml`, sehingga perbedaan pengembangan tidak perlu menduplikasi berkas.

Di Docker Desktop pada Windows, bind mount dari drive Windows melewati lapisan penerjemah dan sangat lambat untuk direktori berisi banyak berkas. Terukur pada stack ini: sekitar 2,0 detik per request dengan `vendor/` di bind mount, turun menjadi sekitar 0,15 detik setelah `vendor/` dipindah ke named volume.

Penempatan vendor/	Waktu per request	Konsekuensi
bind mount Windows	± 2,0 detik	IDE melihat vendor, tapi menjengkelkan saat ngoding
named volume	± 0,15 detik	cepat; salinan di host tidak ikut ter-update
proyek di filesystem WSL2	± 0,15 detik	cepat tanpa efek samping — paling ideal

Analogi — *bind mount Windows seperti mengambil bahan dari gudang di gedung sebelah: setiap langkah kecil harus menyeberang. Named volume adalah rak di dalam dapur itu sendiri.*

Dua pengaturan lain khusus Windows yang wajib ada. Pertama, `opcache.revalidate_freq` jangan diisi 0 di bind mount: nilai itu memaksa pemeriksaan setiap berkas pada setiap request. Kedua, Vite harus memakai polling, karena notifikasi perubahan berkas tidak menembus bind mount.

`src/vite.config.js` — bagian server

```
server: {
  host: '0.0.0.0',
  port: 5173,
  // Tanpa origin, plugin menulis "http://0.0.0.0:5173" ke public/hot
  // dan browser gagal memuat aset.
  origin: 'http://localhost:5173',
  hmr: { host: 'localhost', protocol: 'ws' },
  watch: { usePolling: true, interval: 300 },
}
```

Praktik Terpandu

- [1] Jalankan mode pengembangan (override dimuat otomatis)
`docker compose up -d`
- [2] Ukur waktu respons setelah aplikasi hangat
`curl -s -o /dev/null http://localhost:8080/`
`Measure-Command { curl -s -o /dev/null http://localhost:8080/ }`
- [3] Periksa alamat yang ditulis Vite
`Get-Content src/public/hot`
- [4] Ubah satu teks di `src/resources/views/welcome.blade.php` lalu muat ulang halaman

Yang harus terlihat: berkas hot berisi `http://localhost:5173` (bukan `0.0.0.0`), dan perubahan pada berkas Blade langsung tampak tanpa build ulang.

KESALAHAN UMUM PEMULA

Menyimpan `node_modules` atau `vendor` di named volume dengan penjaga "pasang bila belum ada".
 Dependensi jadi membeku: mengubah `package.json` tidak berpengaruh apa pun sampai volume dihapus manual. Jalankan `npm install` setiap start — ia idempoten dan cepat.
 Memakai image dev dan produksi dengan tag yang sama. Build saat ngoding akan menimpa image produksi dengan versi berisi Xdebug, dan itulah yang kelak ter-deploy.

Hari 3 — Kubernetes: Konsep Inti

TUJUAN PEMBELAJARAN HARI 3

Menjelaskan apa yang tidak bisa dilakukan Compose dan mengapa Kubernetes ada.
 Memetakan tiap konsep Compose ke objek Kubernetes yang setara.
 Menjelaskan hubungan Deployment, ReplicaSet, Pod, dan Service.
 Memisahkan konfigurasi dan rahasia memakai ConfigMap dan Secret.
 Memilih antara Deployment dan StatefulSet berdasarkan sifat data.
 Menyetel startupProbe, readinessProbe, dan livenessProbe dengan tepat.
 Menyusun manifest memakai Kustomize base dan overlay.

3.1 Kenapa Compose Tidak Cukup

Masalah

Stack Compose berjalan baik di satu mesin. Tetapi ketika mesin itu mati, seluruh aplikasi ikut mati. Ketika trafik naik, tidak ada cara menambah kapasitas selain memperbesar mesin. Ketika sebuah container mati karena bug, tidak ada yang menghidupkannya kembali dengan cerdas — dan ketika deploy versi baru, ada jeda saat container lama berhenti sebelum yang baru siap.

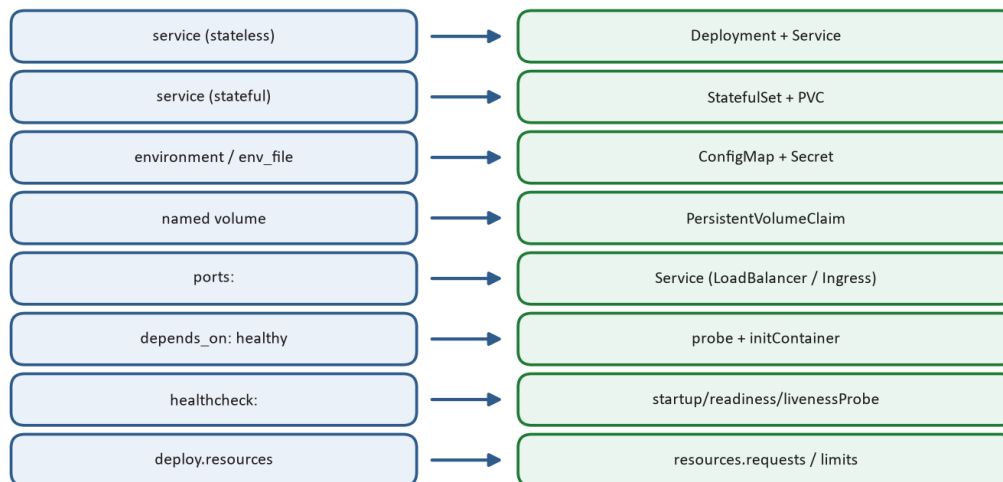
Konsep (sesi konseptual)

Kubernetes adalah sistem yang menjaga keadaan nyata tetap sama dengan keadaan yang diinginkan. Perbedaan cara berpikirnya mendasar. Pada Compose, perintah bersifat imperatif: "jalankan

container ini". Pada Kubernetes, manifest bersifat deklaratif: "harus selalu ada dua salinan aplikasi ini". Kubernetes lalu terus-menerus membandingkan kenyataan dengan keinginan itu dan memperbaiki selisihnya, tanpa diminta lagi.

Perulangan itu — amati, bandingkan, perbaiki — disebut control loop, dan ia yang menjelaskan hampir seluruh perilaku Kubernetes. Pod yang mati akan digantikan. Node yang hilang membuat pod-nya dijadwalkan ulang di tempat lain. Deployment versi baru dilakukan bertahap sambil memastikan jumlah pod sehat tidak pernah turun di bawah batas.

Pemetaan konsep: Compose → Kubernetes



Konsepnya sama; yang berubah adalah siapa yang menjaga keadaan tetap sesuai keinginan

Gambar 7. Hampir semua konsep Compose punya padanan langsung di Kubernetes.

Analogi — Compose adalah satu dapur yang Anda buka dan tutup sendiri setiap hari. Kubernetes adalah jaringan restoran dengan manajer pusat: Anda menetapkan standar — "tiap cabang harus punya dua koki di stasiun panggangan" — dan manajer yang memastikannya terpenuhi, termasuk mencari pengganti saat ada yang tidak masuk.

Kubernetes bukan pengganti Docker. Image yang dibangun pada Hari 1 dipakai apa adanya. Yang berubah hanyalah siapa yang menjalankan dan menjaganya.

KESALAHAN UMUM PEMULA

Menganggap Kubernetes wajib untuk semua aplikasi. Untuk satu aplikasi kecil di satu server, Compose lebih sederhana dan lebih mudah dirawat. Kubernetes membayar dirinya ketika ada banyak service, banyak node, atau kebutuhan rilis tanpa downtime.

Mencoba menghafal seluruh jenis objek Kubernetes sekaligus. Yang dibutuhkan untuk menjalankan aplikasi web hanya segelintir; sisanya bisa dipelajari saat benar-benar dibutuhkan.

3.2 Pod, Deployment, ReplicaSet, dan Service

Masalah

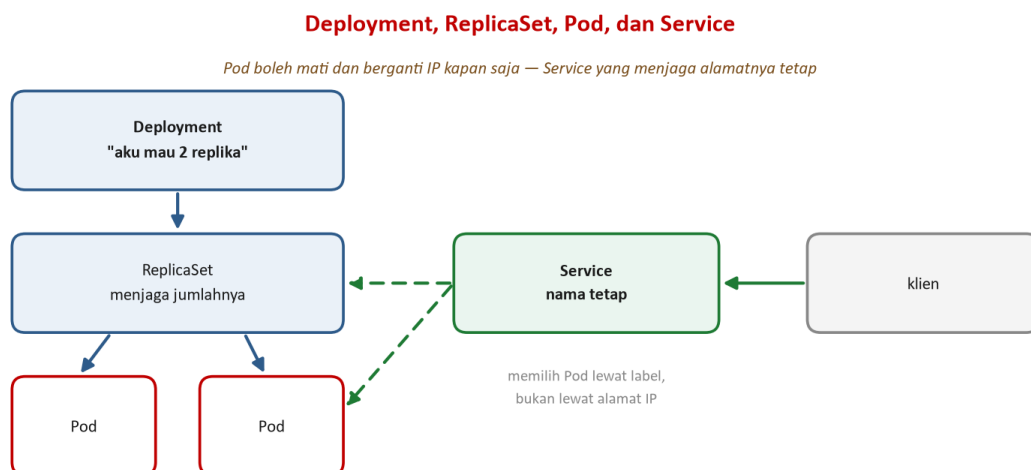
Aplikasi perlu berjalan dalam dua salinan agar tetap melayani saat satu salinan diperbarui. Tetapi tiap salinan punya alamat IP sendiri yang berubah setiap kali pod dibuat ulang. Web server tidak mungkin mengejar alamat yang terus berpindah.

Konsep

Pod adalah unit terkecil yang dijadwalkan Kubernetes. Satu pod memuat satu container atau lebih yang berbagi jaringan dan penyimpanan. Pod bersifat fana — ia dibuat, mati, dan diganti; alamat IP-nya tidak pernah bisa diandalkan.

Deployment menyatakan keinginan: berapa replika, image apa, dan bagaimana cara memperbaruinya. Deployment membuat ReplicaSet, dan ReplicaSet yang menjaga jumlah pod tetap sesuai. Saat deploy versi baru, Deployment membuat ReplicaSet baru dan memindahkan pod sedikit demi sedikit.

Service memberi alamat tetap. Ia memilih pod berdasarkan LABEL, bukan alamat IP, sehingga pod boleh berganti sesering apa pun tanpa mengubah cara memanggilnya. Nama Service menjadi nama host di dalam cluster — persis seperti nama service di Compose.



Gambar 8. Deployment menjaga jumlah; Service menjaga alamat.

Analogi — Deployment adalah SOP tertulis: "stasiun panggangan harus selalu diisi dua koki". Koki boleh berganti shift, sakit, atau diganti orang baru. Service adalah nomor loket yang tertempel di dinding — pelanggan selalu memanggil nomor itu, tidak pernah memanggil nama koki.

k8s/base/app.yaml — Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app.kubernetes.io/name: app      # harus cocok dengan label template
  template:

```

```
metadata:
  labels:
    app.kubernetes.io/name: app    # label inilah yang dicari Service
spec:
  containers:
    - name: php-fpm
      image: laravel-app:latest
      ports:
        - { name: fastcgi, containerPort: 9000 }
```

Nilai di `selector.matchLabels` bersifat IMMUTABLE setelah ter-apply. Mengubahnya memaksa seluruh Deployment dihapus dan dibuat ulang. Karena itu label yang dipakai sebagai selector sebaiknya sesedikit dan sestabil mungkin.

Praktik Terpandu

- [1] Pastikan Kubernetes aktif di Docker Desktop


```
kubectl config current-context
kubectl get nodes
```
- [2] Terapkan manifest


```
kubectl apply -k k8s/overlays/local
```
- [3] Amati hubungan Deployment -> ReplicaSet -> Pod


```
kubectl get deploy,rs,pod -n laravel -l app.kubernetes.io/name=app
```
- [4] Buktikan Kubernetes memulihkan pod yang dihapus


```
kubectl delete pod -n laravel -l app.kubernetes.io/name=app --wait=false
kubectl get pod -n laravel -w
```
- [5] Lihat Service dan endpoint yang dipilihnya


```
kubectl get svc,endpointslice -n laravel
```

Yang harus terlihat: pada langkah [4] pod yang dihapus segera digantikan pod baru dengan nama berbeda, tanpa perintah tambahan.

KESALAHAN UMUM PEMULA

Membuat pod langsung tanpa Deployment. Pod yang mati tidak akan digantikan oleh siapa pun. Label di `selector` tidak sama dengan label di `template`. Deployment akan ditolak, atau lebih buruk, Service tidak menemukan satu pod pun dan semua request menjawab 502.

3.3 ConfigMap dan Secret

Masalah

Konfigurasi tidak boleh tertanam di image — kalau tertanam, satu image tidak bisa dipakai untuk staging dan produksi sekaligus. Password apalagi: ia tidak boleh berada di dalam image maupun di dalam berkas YAML yang masuk ke repositori.

Konsep

ConfigMap menyimpan konfigurasi biasa; Secret menyimpan yang rahasia. Keduanya bisa disuntikkan sebagai variabel environment atau dipasang sebagai berkas.

Perlu jujur soal Secret: isinya hanya dikodekan base64, bukan dienkripsi. Siapa pun yang bisa membacanya lewat API server bisa membaca nilainya. Perlindungan sesungguhnya datang dari RBAC, enkripsi etcd saat istirahat, dan tidak menyimpan berkasnya di repositori.

Analogi — *ConfigMap adalah papan menu yang tertempel di dinding dapur — semua orang boleh membacanya. Secret adalah brankas resep rahasia: terkunci, tetapi kuncinya tetap harus dijaga siapa yang boleh memegang.*

Ada satu jebakan yang sangat sering menimpa pemula. Mengubah isi ConfigMap TIDAK me-restart pod yang memakainya lewat `envFrom`. Nilainya berubah di cluster, tetapi pod yang sedang berjalan tetap memegang nilai lama sampai ada yang me-restart-nya. Manifest terlihat benar, perilakunya tidak — dan itu jenis bug yang sangat sulit dilacak.

Kustomize menyelesaikannya dengan generator: nama ConfigMap diberi imbuhan hash dari isinya. Isi berubah berarti nama berubah, nama berubah berarti spesifikasi Deployment berubah, dan Deployment yang berubah otomatis memicu rollout.

kustomization.yaml — generator ber-hash

```
configMapGenerator:
- name: app-config
  envs:
  - app-config.env

secretGenerator:
- name: app-secrets
  envs:
  - secrets.env          # berkas ini masuk .gitignore
```

Praktik Terpandu

- [1] Lihat nama ConfigMap yang dihasilkan (perhatikan imbuhan hash)
`kubectl get configmap -n laravel`
- [2] Lihat isi ConfigMap
`kubectl get configmap -n laravel -o yaml | Select-String "APP_ENV|DB_HOST"`
- [3] Buktikan Secret hanya base64, bukan enkripsi
`kubectl get secret -n laravel -o jsonpath="{.items[0].data.APP_KEY}"`
(salin nilainya, lalu:)
`[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("<nilai>"))`
- [4] Ubah satu nilai di `k8s/base/app-config.env`, lalu terapkan ulang
`kubectl apply -k k8s/overlays/local`
`kubectl get pod -n laravel -w`

Yang harus terlihat: pada langkah [4] nama ConfigMap berubah dan pod otomatis di-rollout — tanpa perlu restart manual.

KESALAHAN UMUM PEMULA

Menyimpan berkas Secret ke Git "karena toh sudah base64". Base64 bukan enkripsi; siapa pun yang punya akses repositori dapat membacanya dalam hitungan detik.

Memakai ConfigMap biasa lalu bingung kenapa perubahan konfigurasi tidak berpengaruh. Pod harus di-restart, atau pakai generator ber-hash.

3.4 Penyimpanan: PVC, StatefulSet, dan Akses Bersama

Masalah

Pod bersifat fana; apa pun yang ditulis ke dalamnya hilang saat pod diganti. Database jelas tidak boleh begitu. Berkas yang diunggah pengguna juga tidak — dan berkas itu harus bisa dibaca oleh beberapa pod sekaligus.

Konsep

PersistentVolumeClaim (PVC) adalah permintaan penyimpanan: berapa besar, dan mode akses apa. Kubernetes menyediakan volume yang cocok. Mode akses menentukan berapa banyak node yang boleh memasangnya bersamaan.

Mode akses	Arti	Dipakai untuk di stack ini
ReadWriteOnce (RWO)	Dipasang tulis oleh satu node	data MySQL, data Redis
ReadWriteMany (RWX)	Dipasang tulis oleh banyak node	berkas unggahan (app + web server)
ReadOnlyMany (ROX)	Dipasang baca oleh banyak node	aset statis bersama

Deployment cocok untuk workload tanpa identitas: semua pod setara dan boleh diganti. StatefulSet dipakai bila tiap pod perlu identitas dan penyimpanannya sendiri yang tetap. MySQL dan Redis memakai StatefulSet; pod `mysql-0` selalu mendapatkan kembali volume yang sama, bahkan setelah dijadwalkan ulang.

Analogi — *Deployment adalah koki yang boleh siapa saja — semua bisa mengerjakan stasiun yang sama. StatefulSet adalah koki dengan loker pribadi bernomor: koki nomor 0 selalu kembali ke loker nomor 0, karena isi lokernya penting.*

Dua hal khas Docker Desktop wajib disampaikan. Pertama, RWX di sini berhasil hanya karena cluster-nya satu node; di cluster multi-node, hostpath tidak akan bekerja dan diperlukan object storage atau CSI yang benar-benar mendukung RWX. Kedua — dan ini menjebak — provisioner hostpath memetakan volume berdasarkan NAMA PVC. Menghapus PVC tidak menghapus isi direktori di host, dan PVC baru bernama sama akan menemukan data lama masih utuh.

Praktik Terpandu

- [1] Lihat PVC dan PV yang terbentuk

```
kubectl get pvc,pv -n laravel
```
- [2] Buktikan data MySQL bertahan melewati penghapusan pod

```
kubectl exec -n laravel deploy/app -- php artisan migrate:status
kubectl delete pod mysql-0 -n laravel
kubectl wait --for=condition=ready pod/mysql-0 -n laravel --timeout=300s
kubectl exec -n laravel deploy/app -- php artisan migrate:status
```
- [3] Buktikan volume unggahan dipakai bersama app dan web server

```
kubectl exec -n laravel deploy/app -- sh -c "echo halo > storage/app/public/uji.txt"
```



```
curl http://localhost/storage/uji.txt
```

Yang harus terlihat: daftar migrasi tetap sama sebelum dan sesudah pod database dihapus, dan berkas yang ditulis pod app langsung dapat diakses lewat web server.

KESALAHAN UMUM PEMULA

Memakai Deployment untuk database dengan replicas lebih dari satu. Beberapa pod akan menulis ke direktori data yang sama dan merusaknya.

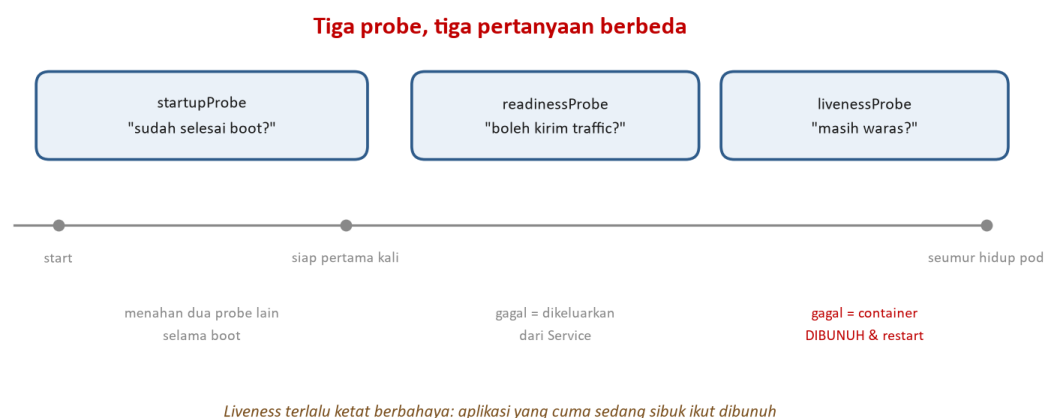
Menyangka `kubectl delete -k` menghapus PVC milik StatefulSet. `volumeClaimTemplates` sengaja meninggalkan PVC-nya; harus dihapus terpisah.

3.5 Probe: Startup, Readiness, dan Liveness

Masalah

Kubernetes perlu tahu tiga hal berbeda tentang tiap container: apakah ia sudah selesai booting, apakah ia siap menerima trafik, dan apakah ia masih waras. Memakai satu pemeriksaan untuk ketiganya menghasilkan salah satu dari dua kegagalan: aplikasi dibunuh saat masih booting, atau aplikasi yang menggantung dibiarkan menerima request.

Konsep



Gambar 9. Tiga probe menjawab tiga pertanyaan yang berbeda.

Probe	Pertanyaan	Akibat bila gagal
startupProbe	Sudah selesai booting?	Menahan dua probe lain; habis jatah = restart
readinessProbe	Boleh dikirim trafik?	Pod dikeluarkan dari Service (tidak dibunuh)
livenessProbe	Masih waras?	Container DIBUNUH dan di-restart

Analogi — *startupProbe* adalah menunggu kompor benar-benar panas. *readinessProbe* adalah tanda "stasiun ini siap menerima pesanan". *livenessProbe* adalah kepala dapur yang, kalau kokinya melamun terlalu lama, memulangkannya dan memanggil pengganti.

Aturan praktis yang penting: *livenessProbe* harus LEBIH LONGGAR daripada *readinessProbe*. Aplikasi yang sedang sibuk sebaiknya dikeluarkan sementara dari load balancer, bukan dibunuh — membunuhnya justru memindahkan beban ke pod lain dan bisa memicu kegagalan berantai.

Sama pentingnya: probe harus menguji fungsi, bukan sekadar port. Pada stack ini php-fpm diuji lewat protokol FastCGI (bukan sekadar port 9000 terbuka), dan MySQL diuji dengan query terautentikasi sebagai user aplikasi — bukan `mysqladmin ping`, yang keluar dengan status sukses bahkan ketika autentikasi ditolak.

k8s/base/app.yaml – tiga probe

```
startupProbe:
  exec: { command: ["fpm-ping"] }
  periodSeconds: 3
  failureThreshold: 40      # beri sampai ~2 menit untuk boot
readinessProbe:
  exec: { command: ["fpm-ping"] }
  periodSeconds: 10
livenessProbe:
  exec: { command: ["fpm-ping"] }
  periodSeconds: 20        # lebih longgar daripada readiness
  failureThreshold: 3
```

Praktik Terpandu

- [1] Lihat konfigurasi probe yang sedang berlaku
`kubectl describe pod -n laravel -l app.kubernetes.io/name=app | Select-String "Probe"`
- [2] Amati *readiness* bekerja saat rollout
`kubectl rollout restart deployment/app -n laravel`
`kubectl get pod -n laravel -w`
- [3] Buktikan pod yang belum Ready tidak menerima trafik
`kubectl get endpointslice -n laravel`

Yang harus terlihat: pod baru berstatus Running tetapi 0/1 selama beberapa saat, dan baru masuk daftar endpoint setelah menjadi 1/1.

KESALAHAN UMUM PEMULA

Menyetel *livenessProbe* terlalu ketat pada aplikasi yang lambat booting, tanpa *startupProbe*. Container dibunuh terus sebelum sempat siap — gejalanya `CrashLoopBackOff` yang tak berujung. Memakai endpoint yang menyentuh database untuk *livenessProbe*. Database yang sedang lambat akan membuat SELURUH pod aplikasi dibunuh bersamaan.

3.6 Kustomize: Base, Overlay, dan Generator

Masalah

Manifest untuk pengembangan dan produksi hampir sama, hanya berbeda pada beberapa nilai. Menyalin seluruh berkas untuk tiap lingkungan berarti setiap perbaikan harus dikerjakan dua kali — dan cepat atau lambat keduanya akan berbeda tanpa disadari.

Konsep

Kustomize sudah menyatu di dalam `kubectl` dan bekerja tanpa templating. Ia memakai manifest YAML biasa sebagai `base`, lalu `overlay` menerapkan perubahan di atasnya. Karena base tetap berupa YAML yang sah, ia bisa dibaca dan divalidasi apa adanya.

Analogi — *base adalah resep induk. Overlay adalah catatan kecil di pinggir kertas: "untuk cabang Bandung, kurangi garam". Resep induknya tidak pernah disalin ulang.*

k8s/overlays/local/kustomization.yaml

```
# k8s/overlays/local/kustomization.yaml
resources:
- ../../base

secretGenerator:
- name: app-secrets
  envs: [secrets.env]

patches:
- target:
    kind: Deployment|StatefulSet|Job
    name: (app|caddy|horizon|scheduler|migrate|mysql)
  patch: |-
    - op: replace
      path: /spec/template/spec/containers/0/imagePullPolicy
      value: Never
```

`imagePullPolicy: Never` dipakai karena image dibangun lokal dan tidak ada di registry mana pun. Bila image belum ada, kegagalannya jelas (`ErrImageNeverPull`) — jauh lebih baik daripada Kubernetes mencoba menarik dari Docker Hub lalu memberi pesan yang membingungkan.

Satu catatan tentang label. Fitur lama `commonLabels` menyisipkan label ke dalam SELECTOR Deployment, padahal selector bersifat immutable. Gunakan bentuk baru `labels` dengan `includeSelectors: false` agar label bisa ditambah tanpa memaksa penghapusan workload.

Praktik Terpandu

- [1] Lihat hasil render TANPA menerapkannya
`kubectl kustomize k8s/overlays/local`
- [2] Simpan hasil render untuk ditinjau
`kubectl kustomize k8s/overlays/local > rendered.yaml`
- [3] Terapkan
`kubectl apply -k k8s/overlays/local`
- [4] Bandingkan apa yang akan berubah sebelum menerapkan
`kubectl diff -k k8s/overlays/local`

Yang harus terlihat: pada hasil render, nama ConfigMap dan Secret sudah berimbuhan hash, dan seluruh objek berada di namespace `laravel`.

CATATAN FASILITATOR

`kubectl diff -k` adalah kebiasaan yang layak ditanamkan sejak hari ini. Melihat apa yang akan berubah sebelum menerapkannya mencegah sebagian besar kecelakaan di produksi.

Bila peserta sudah mengenal Helm, jelaskan bedanya singkat: Helm memakai templating dan punya

konsep rilis; Kustomize tidak memakai templating sama sekali. Keduanya sah — Kustomize dipilih di sini karena tidak menambah perkakas baru.

Hari 4 — Produksi dan Keamanan

TUJUAN PEMBELAJARAN HARI 4

Menerapkan Pod Security Admission tingkat restricted dan membuktikan penegakannya.
Menjalankan seluruh container sebagai non-root dengan rootfs read-only.
Menulis NetworkPolicy dan MENGUJI apakah benar-benar ditegakkan.
Menjalankan migrasi database sebagai Job, bukan di entripoint tiap pod.
Melakukan rilis tanpa downtime memakai rolling update, PDB, dan graceful shutdown.
Membaca kegagalan nyata dari log dan status, lalu menemukan akar masalahnya.

4.1 Pod Security Admission

Masalah

Secara bawaan, Kubernetes mengizinkan pod berjalan sebagai root, meminta hak istimewa, dan memasang direktori host. Satu manifest yang ceroboh — atau satu image dari sumber yang tidak diperiksa — sudah cukup untuk memberi akses jauh melampaui yang dibutuhkan aplikasi.

Konsep

Pod Security Admission adalah pengendali bawaan Kubernetes yang memeriksa setiap pod sebelum dijadwalkan. Ia diaktifkan lewat label pada namespace, dan menyediakan tiga tingkat: **privileged** (tanpa batas), **baseline** (menutup celah yang paling jelas), dan **restricted** (praktik terbaik, paling ketat).

Lapisan keamanan yang dipakai

Namespace · Pod Security Admission: **restricted**

menolak pod yang melanggar SEBELUM dijadwalkan

securityContext · **runAsNonRoot, readOnlyRootFilesystem, drop ALL**

membatasi apa yang bisa dilakukan proses di dalam container

ServiceAccount · **automountServiceAccountToken: false**

container yang jebol tidak punya kredensial ke API server

NetworkPolicy · **default deny + izin eksplisit**

membatasi siapa boleh menghubungi MySQL dan Redis

*Baris terakhir berwarna merah: manifest-nya benar, tetapi CNI bawaan
Docker Desktop TIDAK menegakkannya — sudah diuji langsung*

Gambar 10. Empat lapisan yang dipakai, dan satu yang tidak ditegakkan di Docker Desktop.

Mode label	Perilaku
enforce	Pod yang melanggar DITOLAK
warn	Pod tetap dibuat, peringatan muncul di kubectl
audit	Pelanggaran dicatat di audit log

Analogi — Pod Security adalah pemeriksaan seragam dan sanitasi di pintu masuk dapur. Yang tidak memenuhi syarat tidak diizinkan masuk — bukan ditegur setelah menyentuh makanan.

k8s/base/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: laravel
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

Tingkat **restricted** menuntut empat hal sekaligus dari setiap container: berjalan sebagai non-root, tidak boleh privilege escalation, seluruh capability di-drop, dan memakai seccomp profile RuntimeDefault. Konsekuensinya nyata — image php-fpm resmi menjalankan proses master sebagai root, sehingga konfigurasinya harus disesuaikan agar lolos.

Praktik Terpandu

- [1] Periksa label namespace
`kubectl get ns laravel --show-labels`
- [2] Buktikan penegakannya dengan pod yang sengaja melanggar
`kubectl run pss-uji -n laravel --image=busybox --restart=Never --command -- sleep 30`
- [3] Bandingkan dengan namespace tanpa label
`kubectl create ns coba-bebas`
`kubectl run bebas -n coba-bebas --image=busybox --restart=Never --command -- sleep 30`
`kubectl delete ns coba-bebas`

Yang harus terlihat pada langkah [2] — penolakan yang menyebut keempat pelanggaran:

```
Error from server (Forbidden): pods "pss-uji" is forbidden:
violates PodSecurity "restricted:latest":
  allowPrivilegeEscalation != false
  unrestricted capabilities
  runAsNonRoot != true
  seccompProfile
```

KESALAHAN UMUM PEMULA

Menyetel label hanya **warn** lalu menganggap sudah aman. Peringatan tidak menghentikan apa pun. Menaruh workload di namespace **default**, yang tidak berlabel sama sekali sehingga tidak diperiksa.

4.2 securityContext: Non-root dan Rootfs Read-only

Masalah

Proses yang berjalan sebagai root di dalam container, bila berhasil dieksploitasi, punya keleluasaan jauh lebih besar untuk memasang perkakas, mengubah berkas sistem, dan mencoba keluar dari isolasi. Padahal aplikasi web hanya perlu membaca kodenya dan menulis ke beberapa direktori tertentu.

Konsep

`securityContext` menetapkan batas kemampuan proses. Empat setelan yang paling menentukan:

Setelan	Arti	Alasan
<code>runAsNonRoot: true</code>	Menolak container yang mencoba jalan sebagai root	Membatasi kerusakan bila proses dikompromikan
<code>readOnlyRootFilesystem: true</code>	Sistem berkas container tidak bisa ditulis	Penyerang tidak bisa menanam berkas atau mengubah kode
<code>allowPrivilegeEscalation: false</code>	Proses anak tidak bisa menambah hak	Menutup jalur eskalasi lewat binary setuid
<code>capabilities.drop: [ALL]</code>	Membuang seluruh capability Linux	Aplikasi web tidak butuh satu pun

Analogi — koki diberi kunci untuk stasiunnya saja, bukan kunci seluruh gedung. Dan mejanya tidak bisa dibor atau dipaku — kalau perlu menaruh sesuatu, sudah disediakan rak khusus.

Rootfs read-only menuntut setiap direktori yang memang harus writable dipasang eksplisit. Untuk Laravel ada tiga: `storage` (PVC), `bootstrap/cache`, dan `/tmp`. Kepemilikan berkas di volume diurus `fsGroup`, sehingga proses non-root tetap bisa menulis tanpa perlu `chown` manual.

k8s/base/app.yaml — securityContext

```
securityContext:                # tingkat pod
  runAsNonRoot: true
  runAsUser: 82                 # www-data pada image Alpine PHP
  runAsGroup: 82
  fsGroup: 82
  fsGroupChangePolicy: OnRootMismatch
  seccompProfile: { type: RuntimeDefault }
containers:
- name: php-fpm
  securityContext:              # tingkat container
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities: { drop: ["ALL"] }
  volumeMounts:
    - { name: storage,          mountPath: /var/www/html/storage }
    - { name: bootstrap-cache, mountPath: /var/www/html/bootstrap/cache }
    - { name: tmp,              mountPath: /tmp }
```

Satu efek samping yang perlu diketahui: karena rootfs read-only, direktori home bawaan tidak bisa ditulis dan `php artisan tinker` gagal. Solusinya menyetel `HOME=/tmp`, yang memang sudah tersedia sebagai emptyDir.

Praktik Terpandu

- [1] Buktikan proses berjalan sebagai non-root
`kubectl exec -n laravel deploy/app -- id`
- [2] Buktikan rootfs benar-benar read-only
`kubectl exec -n laravel deploy/app -- sh -c "touch /coba-tulis"`
- [3] Buktikan direktori yang memang perlu writable tetap bisa ditulis
`kubectl exec -n laravel deploy/app -- sh -c "touch storage/logs/uji && echo BISA"`
- [4] Lihat securityContext yang sedang berlaku
`kubectl get pod -n laravel -l app.kubernetes.io/name=app \`
`-o jsonpath="{.items[0].spec.containers[0].securityContext}"`

Yang harus terlihat: langkah [1] menampilkan uid=82, langkah [2] ditolak dengan "Read-only file system", dan langkah [3] berhasil.

KESALAHAN UMUM PEMULA

Menyerah pada readOnlyRootFilesystem karena aplikasi gagal, lalu mematikannya. Yang benar adalah menemukan direktori mana yang perlu ditulis, lalu memasangnya sebagai volume.

Menyetel runAsUser dengan angka sembarang. Angkanya harus cocok dengan pemilik berkas di dalam image, kalau tidak aplikasi tidak bisa membaca kodenya sendiri.

4.3 NetworkPolicy dan Batas Kepercayaan

Masalah

Secara bawaan, setiap pod di cluster dapat menghubungi setiap pod lain. Artinya pod mana pun — termasuk milik tim lain di namespace lain — bisa membuka koneksi langsung ke MySQL dan Redis. Password memang melindungi, tetapi hanya itu satu-satunya lapisan yang tersisa.

Konsep

NetworkPolicy adalah firewall tingkat pod. Polanya: tolak semua lebih dulu, lalu izinkan jalur yang memang diperlukan. Pemilihan pod dilakukan lewat label, sama seperti Service.

Analogi — *kartu akses ruangan. Bukan hanya pintu depan yang dikunci — gudang bahan dan ruang brankas punya kuncinya sendiri, dan tidak semua pegawai memegang semuanya.*

k8s/base/networkpolicies.yaml — inti polanya

```
# Tolak semua lebih dulu
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-all, namespace: laravel }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
# Lalu izinkan yang perlu: MySQL hanya menerima dari workload PHP
```

```
kind: NetworkPolicy
metadata: { name: mysql-ingress, namespace: laravel }
spec:
  podSelector:
    matchLabels: { app.kubernetes.io/name: mysql }
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector:
          matchExpressions:
            - key: app.kubernetes.io/name
              operator: In
              values: [app, horizon, scheduler, migrate]
      ports: [{ protocol: TCP, port: 3306 }]
```

Jangan lupa mengizinkan DNS. Begitu egress ditolak secara bawaan, pod tidak lagi bisa me-resolve nama Service, dan gejalanya terlihat seperti kegagalan aplikasi — bukan kegagalan jaringan.

Hal terpenting dari sesi ini justru bukan cara menulisnya, melainkan kewajiban mengujinya. NetworkPolicy hanya berlaku bila CNI cluster mengimplementasikannya. CNI bawaan Docker Desktop TIDAK. Manifest tetap diterima API server tanpa keluhan apa pun — dan itulah bahayanya: keamanan yang terlihat ada, tetapi tidak menegakkan apa-apa.

Praktik Terpandu

- [1] Terapkan seluruh NetworkPolicy
`kubectl get networkpolicy -n laravel`
- [2] Jalankan pod uji TANPA label yang diizinkan mana pun
`kubectl run netpol-uji -n laravel --image=redis:8-alpine --restart=Never \`
`--overrides='{ "spec": { "securityContext": { "runAsNonRoot": true, "runAsUser": 999, "seccompProfile": { "type": "RuntimeDefault" } } } }' --command -- sleep 120`
- [3] Coba hubungi database dan cache dari pod itu
`kubectl exec -n laravel netpol-uji -- nc -z mysql 3306`
`kubectl exec -n laravel netpol-uji -- nc -z redis 6379`
- [4] Bersihkan
`kubectl delete pod netpol-uji -n laravel`

Yang harus terlihat DI DOCKER DESKTOP: kedua koneksi BERHASIL, membuktikan policy tidak ditegakkan. Di cluster dengan Calico atau Cilium, keduanya akan gagal. Peserta harus menjalankan uji ini di setiap cluster baru sebelum menganggap policy aktif.

KESALAHAN UMUM PEMULA

Menganggap NetworkPolicy aktif hanya karena `kubectl apply` berhasil. Tanpa CNI yang mendukung, ia tidak memblokir apa pun.

Menolak seluruh egress tanpa mengizinkan DNS ke kube-system. Semua resolusi nama gagal, dan gejalanya menyesatkan seperti error aplikasi.

4.4 Migrasi Database sebagai Job

Masalah

Bila migrasi dijalankan di entripoint setiap pod, maka dua replika akan menjalankan migrasi yang sama secara bersamaan saat rollout. Hasilnya bisa berupa deadlock, tabel setengah jadi, atau migrasi yang tercatat berhasil padahal belum selesai.

Konsep

Job menjalankan pod sampai selesai, lalu berhenti. Berbeda dari Deployment yang menjaga pod tetap hidup, Job memberi satu titik eksekusi yang jelas dengan status yang bisa diperiksa — dan kegagalannya terlihat, bukan tersembunyi di antara log pod aplikasi.

Analogi — *Job adalah tugas persiapan sebelum restoran buka — menata ulang gudang. Dikerjakan sekali oleh satu orang, ada penanggung jawabnya, dan hasilnya diperiksa sebelum pintu dibuka.*

k8s/base/migrate-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: migrate
  annotations:
    app.kubernetes.io/revision: "1" # naikan untuk memaksa migrasi ulang
spec:
  backoffLimit: 3
  ttlSecondsAfterFinished: 600
  template:
    spec:
      restartPolicy: Never
      initContainers:
        - name: wait-for-deps
          # Service headless belum punya record DNS sampai ada pod Ready.
          # Tanpa penantian ini, Job kehabisan backoffLimit sebelum DB hidup.
          command: ["sh", "-c", "until php -r '...'; do sleep 2; done"]
      containers:
        - name: migrate
          command: ["php", "artisan", "migrate", "--force", "--isolated"]
```

Dua detail yang mudah terlewat. Pertama, spesifikasi Job bersifat immutable — `kubectl apply` pada Job yang sudah ada akan ditolak; anotasi revisi dipakai untuk memaksa pembuatan ulang. Kedua, `--isolated` menyimpan kunci penguncian di cache store. Itu aman ketika cache berada di Redis, tetapi menjadi lingkaran setan bila cache store adalah database: kuncinya disimpan di tabel `cache_locks`, yang justru baru dibuat oleh migrasi itu sendiri.

Praktik Terpandu

- [1] Lihat status Job
`kubectl get job -n laravel`
- [2] Baca log migrasi
`kubectl logs -n laravel job/migrate`
- [3] Jalankan ulang migrasi dengan sengaja
`kubectl delete job migrate -n laravel`
`kubectl apply -k k8s/overlays/local`
- [4] Periksa hasilnya dari sisi aplikasi
`kubectl exec -n laravel deploy/app -- php artisan migrate:status`

Yang harus terlihat: Job berstatus Complete 1/1, dan seluruh migrasi tercatat Ran.

KESALAHAN UMUM PEMULA

Memakai `--isolated` sementara CACHE_STORE masih database. Pada database kosong, migrasi SELALU gagal karena tabel penguncinya belum ada.

Menyangka Job akan berjalan ulang otomatis setiap deploy. Job yang sudah selesai tidak dijalankan lagi; harus dihapus atau anotasinya diubah.

4.5 Rilis Tanpa Downtime

Masalah

Saat deploy versi baru, pengguna tidak boleh melihat error. Padahal pod lama harus berhenti dan pod baru butuh waktu untuk siap. Tanpa pengaturan, ada jeda ketika request jatuh ke pod yang sudah tidak melayani atau belum siap.

Konsep

Empat mekanisme bekerja bersama:

Mekanisme	Perannya
maxUnavailable: 0	Pod lama baru dihentikan setelah pod baru benar-benar Ready
readinessProbe	Menentukan kapan pod baru dianggap siap menerima trafik
preStop + grace period	Memberi jeda agar pod dikeluarkan dari Service sebelum mati
PodDisruptionBudget	Menjaga jumlah minimum pod sehat saat node dikosongkan

Jeda `preStop` sering dianggap tidak perlu, padahal ia menutup celah nyata. Saat pod dihapus, dua hal terjadi hampir bersamaan: sinyal berhenti dikirim ke container, dan pod dihapus dari daftar endpoint Service. Keduanya tidak berurutan. Tanpa jeda singkat, ada request yang masih dikirim ke pod yang sudah mulai berhenti.

Analogi — *koki yang mau pulang tidak langsung melepas celemek. Ia menyelesaikan pesanan yang sedang dipegang, dan namanya lebih dulu dihapus dari papan penerima pesanan baru.*

k8s/base/app.yaml – strategi rollout

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0      # jangan pernah kurang dari jumlah yang diminta
    maxSurge: 1
...
lifecycle:
  preStop:
    sleep: { seconds: 5 }
terminationGracePeriodSeconds: 30
```

Horizon memerlukan perlakuan berbeda. Ia memakai strategi **Recreate**, bukan **RollingUpdate**, karena dua supervisor yang hidup bersamaan akan berebut antrian yang sama. Scheduler juga wajib **Recreate** dan tepat satu replika — dua scheduler berarti setiap tugas terjadwal berjalan dua kali.

Praktik Terpandu

```
[1] Siapkan pemantau request di satu jendela terminal
    while ($true) { curl -s -o $null -w "$(Get-Date -f HH:mm:ss) %{http_code}`n" `
        http://localhost/up; Start-Sleep 1 }

[2] Di jendela lain, lakukan rollout
    kubectl rollout restart deployment/app -n laravel
    kubectl rollout status deployment/app -n laravel

[3] Hitung berapa request yang gagal

[4] Lihat riwayat rollout dan cara mengembalikannya
    kubectl rollout history deployment/app -n laravel
    kubectl rollout undo deployment/app -n laravel
```

Yang harus terlihat: seluruh request menjawab 200 selama rollout berlangsung. Pada stack ini, pengujian 30 request berturut-turut menghasilkan 0 kegagalan.

KESALAHAN UMUM PEMULA

Memakai **maxUnavailable: 1** pada Deployment dengan 2 replika. Saat rollout, kapasitas turun setengah dan pod yang tersisa bisa kewalahan.

Memakai **RollingUpdate** untuk scheduler. Sesaat akan ada dua scheduler dan tugas terjadwal berjalan ganda.

4.6 Autentikasi dan Gate Dashboard

Masalah

Dashboard Horizon menampilkan seluruh isi antrian, termasuk data di dalam job. Membiarkannya terbuka di produksi sama saja membocorkan data. Tetapi menutupnya rapat-rapat juga membuatnya tidak berguna.

Konsep

Horizon memakai gate otorisasi Laravel. Di **APP_ENV=local** gate dilewati; di lingkungan lain, hanya yang lolos gate boleh masuk. Pola yang baik adalah membaca daftar yang diizinkan dari environment, sehingga menambah orang tidak memerlukan deploy ulang kode.

src/app/Providers/HorizonServiceProvider.php

```
protected function gate(): void
{
    Gate::define('viewHorizon', function ($user = null) {
        $allowed = array_filter(array_map('trim',
            explode(',', (string) env('HORIZON_ALLOWED_EMAILS', ''))));

        return $user !== null
            && $allowed !== []
            && in_array($user->email, $allowed, true);
    });
}
```

Perhatikan bahwa gate memeriksa user yang SEDANG LOGIN. Selama aplikasi belum punya mekanisme autentikasi, nilainya selalu null dan dashboard tetap tertutup — daftar email tidak berpengaruh apa pun. Karena itu starter kit autentikasi dipasang lebih dulu.

Analogi — *daftar tamu di pintu ruang brankas hanya berguna kalau ada resepsionis yang memeriksa identitas. Tanpa resepsionis, daftar itu hanya kertas.*

Satu hal yang wajib disampaikan saat memasang starter kit: ia menambahkan halaman pendaftaran yang terbuka untuk publik. Akun hasil pendaftaran tidak bisa membuka Horizon karena gate memeriksa email — tetapi tetap mendapat akses ke halaman aplikasi. Untuk deployment yang tidak menerima pendaftaran umum, route pendaftaran harus dihapus.

Praktik Terpandu

- [1] Pastikan daftar email sudah terisi di ConfigMap
`kubectl exec -n laravel deploy/app -- printenv HORIZON_ALLOWED_EMAILS`
- [2] Buktikan gate menolak yang belum login
`curl -s -o /dev/null -w "%{http_code}\n" http://localhost/horizon`
- [3] Buat akun yang diizinkan
`kubectl exec -n laravel deploy/app -- php artisan tinker --execute=\n"App\Models\User::updateOrCreate(['email'=>'anda@contoh.com'],\n ['name'=>'Nama', 'password'=>Hash::make('rahasia'), 'email_verified_at'=>now()]);"`
- [4] Login lewat browser di `http://localhost/login` lalu buka `/horizon`
- [5] Daftarkan akun KEDUA dengan email berbeda, lalu coba buka `/horizon`

Yang harus terlihat: langkah [2] menjawab 403; setelah login sebagai email yang diizinkan, `/horizon` menjawab 200; dan akun kedua tetap 403 walaupun sudah login.

KESALAHAN UMUM PEMULA

Mengisi daftar email lalu menganggap selesai, padahal aplikasi belum punya halaman login sama sekali. Gate akan selalu menolak karena tidak ada user yang login.

Membiarkan halaman pendaftaran terbuka di lingkungan internal tanpa disadari.

4.7 Membaca Kegagalan: Studi Kasus Nyata

Masalah

Semua materi sebelumnya menunjukkan konfigurasi yang sudah benar. Di lapangan, yang dihadapi adalah pesan error yang tidak menjelaskan apa pun. Kemampuan membaca kegagalan jauh lebih berharga daripada menghafal manifest yang benar.

Konsep (sesi konseptual)

Empat kasus berikut benar-benar terjadi saat stack ini dibangun. Semuanya punya pola yang sama: pesan error menunjuk ke tempat yang salah.

Kasus 1 — `exec: operation not permitted`

Container Caddy gagal start dengan pesan `exec /usr/bin/caddy: operation not permitted`. Pesan itu terdengar seperti masalah izin berkas, padahal berkasnya bisa dieksekusi. Penyebabnya:

binary Caddy memiliki file capability `cap_net_bind_service=ep` agar bisa membuka port 80 sebagai non-root. Dengan `allowPrivilegeEscalation: false` — yang wajib di Pod Security restricted — kernel menolak meng-exec berkas yang punya effective file capability. Perbaikannya: hapus capability itu saat build, karena port yang dipakai 8080.

Kasus 2 — Host ... is not allowed to connect

Aplikasi ditolak MySQL dengan pesan yang mengarah ke masalah jaringan atau grant. Akar masalahnya jauh berbeda: variabel `MYSQL_PWD` disetel di tingkat container agar password tidak muncul di daftar proses. Tetapi klien `mysql` membaca variabel itu SECARA OTOMATIS — termasuk ketika endpoint resmi MySQL menyiapkan database lewat temporary server yang root-nya masih berpassword kosong. Inisialisasi gagal di tengah, keluar dengan status 1, dan meninggalkan direktori data berisi tabel sistem tanpa user aplikasi. Sesudahnya MySQL start normal selamanya, karena direktorinya sudah tidak kosong.

Pelajarannya ada dua. Pertama, variabel environment bisa punya arti khusus bagi program lain di dalam container yang sama. Kedua, kegagalan yang meninggalkan keadaan setengah jadi jauh lebih berbahaya daripada kegagalan yang bersih.

Kasus 3 — Menghapus PVC tidak menghapus data

Setelah kasus 2, langkah wajar adalah menghapus PVC dan mulai dari nol. Tetapi masalahnya tetap ada. Provisioner hostpath Docker Desktop memetakan volume berdasarkan NAMA PVC; menghapus PVC — bahkan sampai PV-nya hilang — tidak menghapus isi direktori di host. PVC baru bernama sama menemukan data lama masih utuh. Direktori data harus dikosongkan secara eksplisit.

Kasus 4 — Aturan penolakan yang kalah urutan

Konfigurasi Caddy memuat aturan yang menolak akses ke berkas titik. `kubectl apply` berhasil, konfigurasi tervalidasi, tetapi `/.htaccess` tetap tersaji dengan status 200. Penyebabnya: Caddy tidak mengeksekusi direktif menurut urutan penulisan — ada urutan bawaan, dan `file_server` menang atas `respond 403`. Perbaikannya membungkus seluruh handler dalam blok `route`.

Kasus ini paling penting dari sudut pandang keamanan, karena tidak menimbulkan error sama sekali. Satu-satunya cara menemukannya adalah MENGUJI perilakunya, bukan membaca ulang konfigurasinya.

PRINSIP YANG PALING LAYAK DIBAWA PULANG

Konfigurasi yang ter-apply tanpa error belum tentu bekerja. Uji perilakunya.

Pesan error sering menunjuk ke gejala, bukan ke penyebab. Telusuri sampai ke log paling awal — `kubectl logs --previous` dan log container pertama sering menyimpan jawabannya.

Kegagalan yang meninggalkan keadaan setengah jadi lebih berbahaya daripada kegagalan total.

Setiap klaim keamanan harus punya cara pembuktiannya. Bila tidak bisa diuji, anggap belum aktif.

Lampiran A — Pemetaan Silabus ke Sesi

Tabel ini dipakai fasilitator untuk memastikan seluruh butir silabus tersampaikan, dan untuk menyesuaikan alokasi waktu bila durasi pelatihan dipersingkat.

Butir silabus	Sesi	Perkiraan durasi
Konsep containerisasi	1.1	45 menit
Image, layer, dan optimasi build	1.2	60 menit
Arsitektur PHP-FPM dan web server	1.3	45 menit
Multi-stage build	1.4	75 menit
Keamanan image dan build context	1.5	45 menit
Orkestrasi lokal dengan Compose	2.1	60 menit
Database dalam container	2.2	60 menit
Cache dan antrean dengan Redis	2.3	60 menit
Queue worker dan penjadwalan	2.4	60 menit
Inisialisasi runtime (entrypoint)	2.5	45 menit
Pilihan web server	2.6	45 menit
Lingkungan pengembangan	2.7	45 menit
Pengantar orkestrasi Kubernetes	3.1	45 menit
Objek beban kerja dan jaringan	3.2	75 menit
Manajemen konfigurasi dan rahasia	3.3	60 menit
Penyimpanan persisten	3.4	60 menit
Pemeriksaan kesehatan	3.5	45 menit
Manajemen manifest	3.6	45 menit
Kebijakan keamanan pod	4.1	45 menit
Pengerasan container	4.2	60 menit

Segmentasi jaringan	4.3	60 menit
Migrasi basis data terkelola	4.4	45 menit
Strategi rilis	4.5	60 menit
Kontrol akses dashboard	4.6	45 menit
Analisis kegagalan	4.7	60 menit

Lampiran B — Pemetaan Praktik Terpandu ke Latihan Mandiri

Setiap Praktik Terpandu di modul ini punya pasangan latihan di Panduan Peserta. Praktik Terpandu dikerjakan bersama-sama; latihan dikerjakan mandiri tanpa perintah yang sudah jadi.

Sesi (Praktik Terpandu)	Latihan	Fokus latihan
1.2 Cache build	L1.1	Mengurutkan ulang Dockerfile lalu mengukur dampaknya
1.3 PHP-FPM	L1.2	Menjelaskan pembagian hak akses master dan pekerja
1.4 Multi-stage	L1.3	Membuktikan perkakas build tidak ikut ke image akhir
1.5 Build context	L1.4	Menemukan berkas yang seharusnya tidak ikut
2.1 Compose	L2.1	Menambah satu service dan menghubungkannya
2.2 MySQL	L2.2	Membedakan container 'Up' dan database 'siap'
2.3 Redis	L2.3	Menentukan kebijakan eviction untuk kasus yang diberikan
2.4 Horizon	L2.4	Melacak job yang gagal sampai ke penyebabnya
2.5 Entrypoint	L2.5	Menemukan akibat config:cache di waktu yang salah
2.6 Web server	L2.6	Menguji paritas dua web server
2.7 Mode dev	L2.7	Mengukur dampak penempatan vendor/
3.2 Objek K8s	L3.1	Menelusuri Deployment sampai ke Pod dan endpoint

3.3 Konfigurasi	L3.2	Membuktikan perubahan ConfigMap memicu rollout
3.4 Penyimpanan	L3.3	Menguji ketahanan data terhadap penghapusan pod
3.5 Probe	L3.4	Menentukan probe yang tepat untuk tiga skenario
3.6 Kustomize	L3.5	Membuat overlay baru untuk lingkungan staging
4.1 Pod Security	L4.1	Memperbaiki manifest yang ditolak
4.2 securityContext	L4.2	Menemukan direktori yang perlu writable
4.3 NetworkPolicy	L4.3	Menguji apakah policy benar-benar ditegakkan
4.5 Rilis	L4.4	Mengukur kegagalan request selama rollout
4.7 Kegagalan	L4.5	Analisis akar masalah dari log yang diberikan

DUA MODE PENYAMPAIAN

Mode penuh (4 hari): seluruh Praktik Terpandu dikerjakan bersama, lalu latihan mandiri dikerjakan di akhir tiap hari.

Mode padat (2 hari): fasilitator mendemonstrasikan Praktik Terpandu, peserta mengerjakan latihan bertanda L1.3, L2.2, L2.3, L3.2, L4.1, dan L4.3 saja — enam latihan yang paling banyak memuat konsep kunci.

Lampiran C — Berkas Pendukung

C.1 Job Contoh untuk Menguji Antrean

src/app/Jobs/PingJob.php

```
<?php

namespace App\Jobs;

use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Queue\Queueable;
use Illuminate\Support\Facades\Log;

class PingJob implements ShouldQueue
{
    use Queueable;

    public function handle(): void
    {
        Log::info('PingJob diproses oleh queue worker.', [
            'hostname' => gethostname(),
        ]);
    }
}
```



```
}  
}
```

C.2 Perintah yang Paling Sering Dipakai

Keperluan	Perintah
Menjalankan stack lokal	<code>docker compose up -d --build</code>
Melihat status dan kesehatan	<code>docker compose ps</code>
Melihat log satu service	<code>docker compose logs -f app</code>
Menjalankan artisan	<code>docker compose run --rm artisan migrate</code>
Menambah paket PHP	<code>docker compose run --rm composer require nama/paket</code>
Menghapus stack beserta datanya	<code>docker compose down -v --remove-orphans</code>
Menerapkan manifest	<code>kubectl apply -k k8s/overlays/local</code>
Melihat pod	<code>kubectl get pods -n laravel</code>
Melihat log pod	<code>kubectl logs -f -l app.kubernetes.io/name=app -n laravel</code>
Log container SEBELUM restart	<code>kubectl logs <pod> -n laravel --previous</code>
Menjalankan artisan di cluster	<code>kubectl exec -n laravel deploy/app -- php artisan ...</code>
Melihat perubahan sebelum apply	<code>kubectl diff -k k8s/overlays/local</code>
Rollout dan memantaunya	<code>kubectl rollout restart deployment/app -n laravel</code>
Mengembalikan rollout	<code>kubectl rollout undo deployment/app -n laravel</code>
Menyelidiki pod bermasalah	<code>kubectl describe pod <pod> -n laravel</code>

C.3 Urutan Diagnosis Saat Pod Bermasalah

Urutan ini menghemat banyak waktu karena bergerak dari informasi paling ringkas ke paling rinci.

- ```
[1] kubectl get pods -n laravel
 -> lihat STATUS dan RESTARTS

[2] kubectl describe pod <pod> -n laravel
```

-> baca bagian Events di paling bawah  
-> baca lastState untuk tahu penyebab terminasi sebelumnya

- [3] `kubectl logs <pod> -n laravel`  
-> log container yang SEKARANG berjalan
- [4] `kubectl logs <pod> -n laravel --previous`  
-> log container SEBELUM restart; sering di sinilah jawabannya
- [5] `kubectl logs <pod> -n laravel -c <initContainer>`  
-> bila pod punya initContainer
- [6] `kubectl exec -n laravel <pod> -- sh`  
-> masuk ke dalam container bila masih hidup

### SELAMAT

Peserta yang menyelesaikan empat hari ini sudah membawa satu aplikasi Laravel dari berkas di laptop sampai berjalan di Kubernetes dengan postur keamanan yang ketat — dan, yang lebih penting, tahu alasan di balik setiap keputusannya.

Langkah lanjutan yang wajar: TLS dengan cert-manager, Ingress, observability (metrics dan tracing), autoscaling, serta pipeline CI/CD yang membangun image dan menerapkan manifest secara otomatis.